
Semantic Retrieval: A Process-Aware Architecture and Retrieval Comparison for Agent Orchestration

Dominik Neumaier^{*1}

¹University of Applied Sciences Karlsruhe & University of Strasbourg

Correspondence: nedo1011@h-ka.de

Abstract

LLM-based agents are increasingly used to automate enterprise business processes by planning and executing tasks across heterogeneous systems. Integrating agents that operate over these systems autonomously requires a semantic layer that lets them interpret available resources and their capabilities. Existing frameworks approximate this by wrapping resources in free-text descriptions and relying on the LLM to reason over them, which gives limited grounding for disambiguating similar resources. This paper takes a different path, asking whether a structured metadata description layer improves selection over free text descriptions and whether semantic enrichment improves it further. To this end, it contributes (1) a process-aware reference architecture that integrates LLM-based agents into existing enterprise process orchestration through three orchestration modes; (2) seven retrieval strategies over a typed Open Resource Discovery (ORD) layer, spanning the embedding, LLM-guided, graph, and agentic retrieval families; and (3) a systematic comparison of these strategies, evaluated on 110 runtime cases from ORD-Bench using semantically enriched and non-enriched resource descriptions. The central finding is that semantic enrichment is *conditional*: it improves selection by up to 15 percentage points for methods that consume typed fields, yet leaves free-text methods flat or degrades them, contradicting the common assumption that richer descriptions uniformly help. The process-aware architecture enables reliable resource selection across the full process spectrum, and an agentic hybrid combining embedding recall with typed disambiguation reaches the highest R@1 (45%). Recall is high across all methods (R@5 up to 70%), so the correct resource is almost always retrieved in the top five, and the remaining challenge is ranking it first.

1. Introduction

Enterprise process automation has traditionally relied on workflow engines, integration platforms, and rule-based systems. Large Language Models (LLMs) introduce a complementary reasoning and planning capability, enabling agent systems to execute process steps by autonomously retrieving information, invoking APIs, and coordinating across heterogeneous enterprise landscapes [1]. As a result, business process automation is evolving towards agent-driven orchestration, in which software agents independently determine which enterprise resources to select and invoke in order to fulfil a business objective [2].

This shift introduces a fundamental resource selection problem. As agents operate across increasingly diverse enterprise landscapes, they must identify the resource that best fulfils a given business intent from many functionally similar alternatives. A single business domain such as procurement or finance may expose hundreds of resources with overlapping names and descriptions, making it difficult to distinguish the correct resource and leading to unreliable runtime selection [3]. This challenge is further compounded by the semantic gap between user intent and resource specifications. Business requests are expressed in domain terminology, whereas resources expose operational descriptions with limited semantic grounding [4]. Consequently, agents must bridge not only ambiguity among similar resources but also the mismatch between business-level intent and implementation-level resource descriptions. Furthermore, because agents cannot replace existing automation, they

must coexist with established workflows and knowledge-intensive processes [5, 6]. This requires mechanisms that allow agents to understand, discover, and select available enterprise resources within these environments.

Current agent frameworks address this primarily through free-text capability descriptions of resources that models interpret at runtime, as exemplified by protocols such as MCP and A2A [7, 8]. However, these descriptions do not constitute a shared semantic layer: they lack a common vocabulary, structured meaning, and consistent criteria for distinguishing functionally similar resources [9]. Consequently, reliable resource selection remains dependent on the model’s interpretation of ambiguous textual signals. Overcoming this limitation requires a uniform foundation that enables agents to autonomously discover and select suitable resources by representing not only descriptive information but also semantic characteristics of enterprise resources [10]. Because these capabilities are invoked to execute business processes, this foundation must also be process-aware and operate across the full spectrum from structured workflows to loosely framed knowledge work.

Following the Design Science Research paradigm of Österle et al. [11], this work develops and evaluates an approach to enable reliable resource selection in heterogeneous enterprise landscapes. The research question is: *How can a process-aware orchestration architecture and semantic resource descriptions enable resource selection by LLM-based agents in heterogeneous enterprise landscapes?*

This question is addressed through three contributions: (1)

a *process-aware reference architecture* that spans the full process-structuredness spectrum through three orchestration modes, enabling the agent layer to coexist with established processes; (2) seven *retrieval strategies* covering embedding-based, LLM-guided, graph-based, and agentic retrieval approaches, operating over a typed description layer to improve resource disambiguation in dense action spaces; and (3) a *systematic comparison of these strategies with and without semantic enrichment*. The description layer is instantiated using Open Resource Discovery (ORD) as the resource description standard. The proposed strategies are evaluated on ORD-Bench, which provides controlled retrieval test cases over enriched and non-enriched resource descriptions, enabling a systematic comparison of the impact of semantic enrichment. All artefacts are published in the repository.¹

Three scope decisions bound the study and are central to interpreting its causal claims. The artefact addresses resource selection but not resource execution; therefore, API invocation is outside the scope. All models are used off the shelf without fine-tuning, allowing observed differences to be attributed primarily to the description layer rather than to model adaptation. Finally, the generalisability of the results is bounded by the assumptions embedded in ORD-Bench, which represents a resource landscape through ORD as description standard with a shared entity-type vocabulary.

2. Related Work

Reliable resource selection depends on four interacting layers, surveyed here according to their role in enabling agent-based execution: the tool-calling mechanisms through which agents invoke resources, the description layer that specifies available capabilities, the semantic technologies that provide business meaning to these descriptions, and the orchestration architectures that embed these capabilities within enterprise processes. Each subsection examines how prior work addresses these layers and how remaining gaps motivate the design choices of this paper.

Tool calling and retrieval. LLM-based agents differ from plain language models by their ability to perceive their environment and act upon it in pursuit of a goal, a defining property of agency in AI research [12, 13]. In enterprise settings, this interaction is enabled through *tool calling*, which allows an agent to invoke external resources, receive their results, and incorporate this information into subsequent reasoning [14, 15]. The invocable resources form an *action space* that the agent must navigate. A common approach is to provide the model with free-text descriptions of available tools in the prompt, allowing selection among them at generation time. Such eager catalogues do not scale: as they grow, reliable selection becomes harder and the likelihood of invalid calls increases [16].

A standard response to similar scaling problems is to ground the model in external context retrieved at inference time, as in retrieval-augmented generation over text [17] or graph-

structured variants over entity knowledge graphs. The same idea transfers to tool calling once the resources themselves are treated as retrievable units: rather than reasoning over the full catalogue, the agent retrieves a task-relevant subset for the current request. Beyond single-pass retrieval, patterns such as ReAct interleave reasoning, retrieval, and invocation so that intermediate results guide subsequent actions over multiple steps [18]. This shift from eager catalogues to retrieval is the mechanism this work builds on.

Existing approaches can be grouped into three retrieval families. *Embedding-based* retrieval ranks resources by cosine similarity between the embeddings of their descriptions and the query; Gorilla introduced this textual baseline over free-text API documentation [16]. However, this approach loses discriminative power once many resources carry unprecise descriptions, as Shi et al. confirm across 43k real-world tools, where even strong retrievers place fewer than one in three in their top-ten [19].

LLM-guided pre-selection instead narrows the search space through hierarchical decomposition over categories or namespaces before selecting a concrete resource; ToolLLM organises 16,000+ APIs into a category tree the model traverses [20], which is particularly effective in large, categorically structured landscapes but adds reasoning steps and inference overhead [1].

Graph-based retrieval models the landscape as a graph and traverses explicit relationships rather than ranking by similarity, allowing it to distinguish resources that read alike but occupy different structural positions; Agent-as-a-Graph represents agents and tools as nodes for multi-hop reasoning [21, 22], building on the broader GraphRAG paradigm [23].

Resource description and registry. Retrieval quality depends not only on the retrieval mechanism but also on the information available about the resources being retrieved. In enterprise landscapes, this information is traditionally provided through registries, where resource providers publish descriptions and consumers discover suitable capabilities at runtime. Making software capabilities discoverable and selectable without human mediation has therefore been a longstanding goal in service-oriented architectures [24]. Early registry-based approaches such as UDDI and WSDL established this pattern by describing how services could be located and invoked, while later approaches such as REST and OpenAPI continued to standardise machine-readable interfaces [25–28]. However, these descriptions remained primarily syntactic: they specified invocation details rather than the business meaning required to distinguish between functionally similar capabilities.

The LLM era inherits rather than closes this gap. Emerging agentic protocols such as MCP and A2A attempt to move beyond purely syntactic interfaces by introducing richer capability descriptions for autonomous discovery and selection. However, these descriptions are typically free-form and lack a shared vocabulary, structured semantics, and consistent criteria for distinguishing functionally similar resources [7, 8]. Hasan et al. further document systematic

¹<https://github.com/tools.sap/I750252/MasterThesis>

“description smells” in real MCP servers that measurably distort agent decisions [9]. Beyond individual agentic protocols, emerging agentic registries and discovery initiatives address resource discovery at broader scopes. The Open Semantic Interchange (OSI) targets analytics and BI metadata [29], while Google’s Agentic Resource Discovery (ARD) focuses on cross-organisational discovery of agent capabilities [30]. However, these approaches do not yet provide a unified semantic description layer covering the heterogeneous resource landscape of an enterprise. At the same time, recent retrieval research indicates that enriching resource descriptions with typed semantic fields can improve retrieval performance over flat text representations [3, 4].

This work therefore builds on Open Resource Discovery (ORD), chosen because it operates at the intra-enterprise level and covers APIs, agents, data products, and entity types in a single harvested landscape model, governed under Linux Foundation Europe as part of the Apeiro Reference Architecture [31, 32]. Its typed and extensible schema supports structured resource selection and semantic augmentation without requiring changes to the underlying specification, making it a suitable foundation for evaluating retrieval strategies under different description conditions. On top of ORD, the strategies are evaluated on ORD-Bench, a benchmark providing a typed enterprise resource landscape with controlled retrieval test cases over semantically enriched and non-enriched description states. A summary of the ORD schema and its properties is given in Appendix B.1.

Semantic technologies and Semantic BPM. Semantic technologies aim to make information machine-interpretable through shared representations of concepts, entities, and their relationships. Different approaches have been proposed along this spectrum, ranging from taxonomies and ontologies that provide structured conceptual models to knowledge graphs that operationalise these models through connected entities and relations [33, 34]. The Semantic Web proposed to make Web content processable by software: since the original Web was designed for human consumption, publishers were asked to add a moderate amount of structure so that machines could perform deterministic operations on the resulting data [35]. Semantic Web Services (SWS) extended this idea to service descriptions by augmenting syntactic interfaces with semantics, enabling automated discovery and invocation [10].

Semantic BPM (SBPM) carried the same ambition into process management. It targeted a long-standing bottleneck: although most processes already reside in enterprise software, translating business intent into system behaviour remained manual analyst-and-developer work [36]. SBPM aimed to automate this translation by representing both business processes and the services realising them on a shared semantic layer, with the vision that executable compositions could be derived from business intent rather than manually engineered [36, 37]. However, SBPM saw limited industrial adoption. Frameworks such as OWL-S and WSMO required substantial expertise to create formal service descriptions, assumed stable ontology environments that con-

Table 1: Representative works against four criteria. (1) typed description layer, (2) process-derived semantic enrichment, (3) process-type-aware routing, (4) empirical retrieval comparison. • = met, ◦ = partial, – = not met.

Work	(1)	(2)	(3)	(4)
OWL-S / WSMO [10, 38]	◦	–	–	–
SBPM [36, 37]	◦	◦	◦	–
OpenAPI / MCP [7, 26]	◦	–	–	–
ORD [31]	•	–	–	–
Gorilla / ToolLLM [16, 20]	–	–	–	◦
ToolGen [41]	–	–	–	◦
Graph retrieval [21]	–	–	–	◦
A-BPMS / Skills [2, 42]	–	–	◦	–
This work	•	•	•	•

flicted with decentralised enterprise ownership models, and faced scalability challenges [38–40].

The resulting lesson is that semantic layers must remain lightweight, maintainable by resource owners, and derivable from existing enterprise artefacts rather than requiring extensive formal modelling. This challenge re-emerges in the era of LLM-based agents: while agents revive the vision of connecting business intent with executable resources, they can reason over heterogeneous representations of capabilities, ranging from unstructured descriptions to structured semantic models. This shifts the focus from defining a complete formal ontology towards designing resource descriptions that provide sufficient structure and context for reliable selection. This motivates ORD’s approach of combining typed structure with semantically extensible descriptions and allows support for multiple retrieval mechanisms.

Orchestration and Agentic BPM. Orchestration describes the coordination of activities and resources through a controlling logic that determines which capability is invoked, when it is executed, and how results are combined. In contrast to choreography, where independent peers coordinate through shared interaction protocols, orchestration relies on a central coordinator that drives execution flow [24]. In agentic systems, this role shifts from deterministic workflow engines towards LLM-based planners that dynamically select and invoke resources.

Effective orchestration depends not only on planning but also on discovering, interpreting, and selecting suitable resources in their process context. This connects orchestration with the preceding foundations of resource descriptions, retrieval mechanisms, and semantic grounding. Since enterprise processes range from structured BPMN workflows to loosely framed, knowledge-intensive CMMN cases where activities emerge at runtime from the case state [5, 6, 43–45], orchestration approaches must account for different degrees of process structuredness.

Recent BPM research explores how LLM-based agents support this spectrum: Rosemann et al. describe a shift from transaction to conversation, automation to autonomisation,

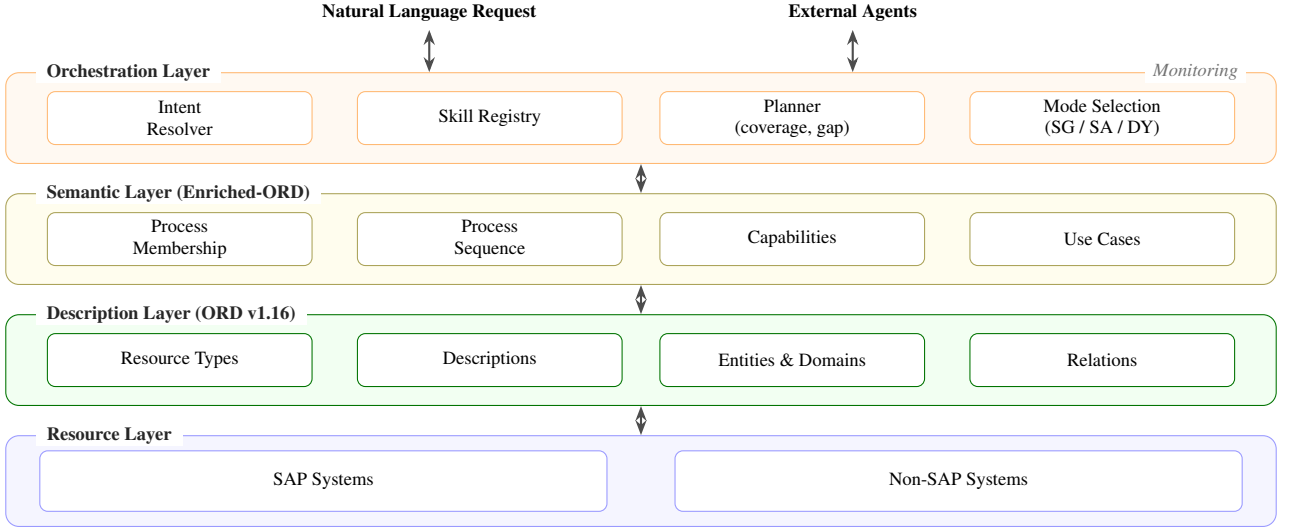


Figure 1: Reference architecture. The Resource Layer contains heterogeneous enterprise systems. The Description Layer aggregates their ORD metadata. The Semantic Layer extends it with four typed process-derived field groups at design time. The Orchestration Layer resolves requests via Intent Resolver, Skills, Planner, and Mode Selection.

and simplification to sophistication [46], while further work positions agents within the implementation phase of the BPM lifecycle [47–49]. Architectures such as A-BPMS structure agentic execution across data, process-intelligence, action, orchestration, and interaction layers [2]; related work further refines planning, execution, and quality control [50] and represents procedural knowledge as modular artefacts through Skills [42, 51, 52]. However, these approaches largely assume that available resources can already be identified and selected reliably, leaving semantic resource selection unresolved.

Research gap. Taken together, these strands converge on a common open question: existing work does not yet provide an integrated approach that combines (1) a typed resource description layer, (2) process-derived semantic enrichment, (3) process-type-aware routing across the full process spectrum, and (4) an empirical comparison of retrieval strategies across both design-time and runtime scenarios. Table 1 maps representative works against these criteria.

3. Methodology

This section presents how the three contributions are constructed. Following the Design Science Research paradigm of Österle et al. [11], their development iterates between design and evaluation: (1) the *reference architecture*, which defines a process-aware orchestrator over a typed, semantically enriched resource description layer; (2) the *retrieval strategies*, which specify seven methods spanning the retrieval families identified in Sec. 2; and (3) the *evaluation setup*, which compares these strategies on ORD-Bench under different description conditions.

Design objectives. The reference architecture is derived from four design objectives that capture the capabilities required for reliable resource selection in heterogeneous enterprise landscapes. These objectives are identified in two stages. First, a systematic analysis of the literature

reviewed in Sec. 2 yields an initial requirement catalogue (Appendix A.1). Second, these literature-derived requirements are validated and refined through five semi-structured expert interviews with practitioners who develop or operate enterprise orchestration systems; the interview transcripts are documented in Appendix A.2. The interview data are analysed using Mayring’s qualitative content analysis [53], with the statement-level coding recorded in Appendix A.3.

Consolidating both stages yields the final requirement catalogue for the reference architecture (Appendix A.4), grouped into four design objectives: *DO1*—process execution across the full structuredness spectrum; *DO2*—a typed, vendor-neutral resource description layer; *DO3*—semantic enrichment linking business processes to resources; and *DO4*—an orchestration mechanism capable of disambiguating functionally similar resources.

Reference architecture. The reference architecture (Fig. 1) addresses the design objectives by introducing a process-aware orchestrator. It is structured into four layers.

The *Resource Layer* represents the existing heterogeneous enterprise landscape comprising SAP and non-SAP systems across ERP, HR, manufacturing, and other business domains. It reflects the operational systems found in today’s enterprises, which continue to expose their native capabilities unchanged. The proposed architecture does not replace or modify these systems. Instead, it builds upon them, treating their capabilities as the resources selected and invoked by the orchestration layer.

The *Description Layer* provides the common representation required to discover and distinguish heterogeneous enterprise resources across system boundaries. Enterprise systems expose system-specific interfaces and metadata that cannot be interpreted uniformly without a shared description format. It therefore represents these resources through Open Resource Discovery (ORD) v1.16 [31]. Each

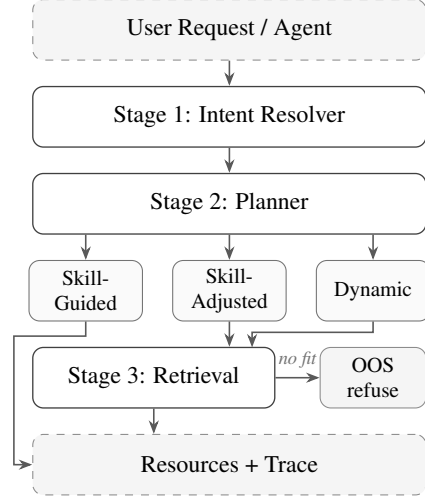


Figure 2: Orchestration modes. The Planner routes each request to one of three modes; Skill-Guided bypasses retrieval (its resources are pinned in the skill), while Skill-Adjusted and Dynamic resolve unbound steps through Stage 3. When no resource fits, Stage 3 abstains (out-of-scope).

system publishes metadata via the read-only ORD Service Provider Interface, which an ORD Aggregator periodically harvests into a single enterprise-wide registry consumable by both AI and conventional applications. Every resource is described through ORD’s standard metadata, including resource types, descriptions, entities and domains, and relations. These typed fields provide structural context for resource discovery and initial disambiguation, while ORD additionally specifies the technical information required for invocation, including API protocols, endpoints, and access details. This layer realises *DO2* by providing a typed, vendor-neutral description layer across heterogeneous enterprise systems. Appendix B.1 provides further details on the ORD specification and its resource description model.

The *Semantic Layer* extends the unified description layer with the business context required for reliable resource selection. While ORD provides structural context that helps distinguish resources, it does not capture the process situations and business intents in which a resource represents the appropriate capability. The semantic layer therefore enriches each resource with additional metadata describing its business meaning and applicability. In this work the enrichment is *process-derived*: each enterprise process model from ORD-Bench is matched to the resources realising its steps, and four typed fields (capabilities, use cases, process groups, and process-successor links) are derived from that mapping and written back into each resource. Process models are one possible source; other enterprise artefacts such as organisational models or knowledge graphs could supply the same layer depending on the context available. This layer realises *DO3* by linking business processes to resources through this process-derived enrichment.

The *Orchestration Layer* performs runtime resource selection by resolving a natural-language request against available enterprise capabilities and the current context. Rather

Table 2: Retrieval methods. Baseline is the no-retrieval reference; Agentic Hybrid is the composite. Full detail in Appendix B.3.

Name	Core mechanism
Free-text Baseline	LLM call over all resource descriptions
Embedding Cosine	Cosine similarity over description fields
Progressive Disclosure	Multi-stage LLM filtering
Graph Walk	ORD graph traversal with LLM reranker
Agentic Tools	Agent with typed ORD tools
Agentic Raw	Agent with access over ORD files
Agentic Hybrid	Agentic loop invoking Embedding, Progressive, and Tools

than reasoning directly over system-specific interfaces, it operates on structured resource descriptions and their semantic enrichment to select the resource best suited to execute the requested capability. The layer comprises four components. The *Intent Resolver* identifies whether the request matches available context artefacts in the Skill Registry and returns the corresponding skill identifier, or `null` if no matching skill exists. The *Skill Registry* stores Markdown artefacts that provide machine-readable business context for the LLM. These artefacts may contain process knowledge, execution guidance, or other domain-specific information that supports context-aware orchestration. A representative skill artefact is shown in Appendix B.2. The *Planner* is invoked when a matching skill exists and determines whether the request is fully covered or requires additional resource discovery. *Mode Selection* then routes execution to one of three orchestration modes. This layer jointly realises *DO1* through process-aware execution across the full structuredness spectrum and *DO4* through runtime selection and disambiguation among functionally similar resources.

Orchestration modes. The orchestrator supports three execution modes covering the full process-structuredness spectrum [5, 6]. Figure 2 illustrates how requests flow through the architecture into each mode. The modes describe different interaction patterns between the components of the orchestration layer based on the availability of reusable context and the required degree of runtime resource discovery. *Skill-Guided* (SG) applies when a suitable skill artefact provides sufficient context for execution: the required resources are specified upfront and the request can be handled without retrieval. *Skill-Adjusted* (SA) applies when available context covers only part of the request; the orchestrator follows the guidance where applicable and uses retrieval to resolve uncovered steps. *Dynamic* (DY) applies when no suitable skill exists and resources must be discovered.

Retrieval methods. When the orchestration layer cannot resolve a request entirely from available context, it must discover suitable resources at runtime (Fig. 2, Stage 3). Retrieval therefore operates over the structured resource descriptions and their semantic enrichment provided by the description and semantic layers. The retrieval methods cover the three retrieval families identified by Xu et al. [1], embedding-based ranking, LLM-guided pre-selection,

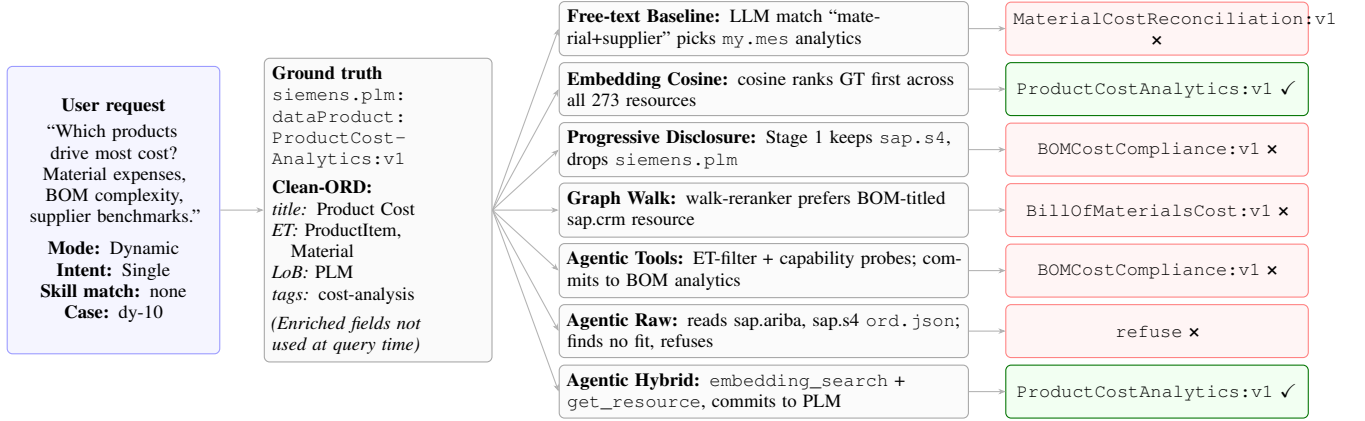


Figure 3: Worked example on one Dynamic case (dy-10, single-intent, clean state): the user request and its ground-truth resource (left), and the resource each of the seven methods commits to (right). Embedding and Agentic Hybrid recover the correct `siemens.plm` resource; Progressive, Graph, and Agentic Tools lock onto BOM-titled distractors in adjacent namespaces; Agentic Raw reads two ERP systems and refuses. The single-case contrast previews the per-method behaviour that the aggregate results in Sec. 4 quantify.

and graph-based traversal, complemented by three agentic variants and a baseline approach (Tab. 2).

All seven methods operate over the same ORD resource descriptions; they differ only in which ORD fields they read and how. The *Baseline* issues a single LLM call over a flat list of resources (`ordId`, `title`, `shortDescription`) without retrieval, serving as the free-text lower bound. *Embedding* ranks resources by cosine similarity between the query and the ORD text fields (`title`, `shortDescription`, `description`), extended on enriched ORD by the appended semantic fields (`partOfGroups`, `processNext`, `capabilities`, `useCases`). *Progressive* narrows the candidate pool through multi-stage LLM filtering over the typed ORD fields `namespace`, `resource modality`, and `entityType`. *Graph* walks the typed ORD relations (shared entity types, `partOfGroups`, integration dependencies, `processNext`) from anchor resources and re-ranks the neighbourhood with an LLM. The three agentic variants give an LLM agent a tool loop over the landscape: *Agentic Tools* exposes typed ORD query tools (filter by entity type, line of business, capability), while *Agentic Raw* exposes only raw access to the ORD JSON files. Finally, *Agentic Hybrid* composes Embedding, Progressive, and Agentic Tools as callable tools and selects among them per query. Full detail for each method is in Appendix B.3.

Orthogonal to which mode invokes it, each strategy must also determine whether a suitable resource exists at all. This *out-of-scope* (OOS) condition is not an execution mode but a rejection capability implemented per strategy (a similarity threshold, an explicit `refuse` action, or confidence-based filtering) and is evaluated separately from the three modes.

Evaluation setup. The architecture and retrieval methods were designed against the requirements and design objectives derived above, which define an ORD-based typed description and semantic layer as the retrieval substrate. ORD-Bench instantiates exactly this substrate: each of its components populates one layer of the reference architec-

Table 3: ORD-Bench at a glance. Top: the landscape properties that populate each architecture layer. Bottom: the 110 runtime test cases per mode. Every case is run on both the clean and the semantically enriched description state, giving a 7×2 (method $\times$ state) design; the 240 ORD-Bench design-time test cases are evaluated in Appendix D.

Landscape (populates the architecture layers)		
Systems	10	Resource Layer
ORD resources	273	Description Layer
Process models	30	Semantic Layer
Skill artefacts	30	Skill Registry (SKILL.md)
Runtime test cases (exercise the orchestration layer)		
Skill-Guided (SG)	30	Routing to a matched skill
Skill-Adjusted (SA)	20	Retrieval for gap steps in a skill
Dynamic (DY)	40	Ad-hoc retrieval, no matching skill
Out-of-Scope (OOS)	20	Refusal when no resource fits
Total	110	$\times 2$ states = 220 runs per method

ture, and its runtime test cases exercise the three orchestration modes plus out-of-scope refusal (Tab. 3).

The four ORD-Bench test case families match the orchestration modes one-to-one, so that together they span the full process-structuredness spectrum the architecture targets: Skill-Guided cases correspond to fully-specified processes (a matching skill covers the request), Skill-Adjusted cases to partially-specified processes (a skill applies but leaves gap steps for retrieval), Dynamic cases to unstructured requests (no skill matches), and Out-of-Scope cases to requests no resource can satisfy. Evaluating each family therefore tests exactly the mode designed for that degree of process structure. Because the benchmark is built natively on ORD and ships every resource in both a clean and a semantically enriched state over the same landscape, the effect of semantic enrichment on retrieval can be isolated.

Four properties make ORD-Bench suitable for this study:

each case targets a resource surrounded by semantically similar distractors, preventing methods from succeeding through trivial matching; the two description states differ only in the four semantic field groups, allowing measured performance differences to be attributed to enrichment rather than changes in the resource landscape; every distractor was placed adversarially using an explicit ambiguity metric rather than sampled at random, so difficulty is a controlled property of the benchmark rather than an artefact of corpus composition; and a dedicated out-of-scope condition tests each method’s ability to abstain when no resource fits, covering the full range from positive retrieval to justified refusal. Figure 3 walks one representative ORD-Bench test case through all seven retrieval methods to make the mechanisms concrete before the aggregate results in Sec. 4. The design-time process-matching flow that produces the enriched state and its retrieval results is documented in Appendix B.4 and Appendix D, with a representative design-time trace provided in Appendix C.1.

4. Results

The four-layer architecture of Sec. 3 was implemented and evaluated on the 110 runtime test cases of ORD-Bench (Tab. 3). The test cases were executed as the architecture prescribes: each request is first routed to the orchestration layer, which then invokes retrieval where the mode requires it. Each case is run on both the clean and the semantically enriched ORD state, so the effect of semantic enrichment is measured per method. The results are then reported by concern: the architecture instantiation, orchestration routing, retrieval strategies, effect of semantic enrichment, out-of-scope refusal, and finally the statistical reliability of the comparison. The 240 ORD-Bench design-time test cases were additionally run to populate the enriched state; their retrieval results corroborate the runtime ranking at a larger sample size and are reported in Appendix D.

Architecture instantiation. Before reporting the test outcomes, it is worth stating how the reference architecture designed in Sec. 3 was turned into the concrete system under evaluation, with each layer populated from an ORD-Bench artefact. The *Resource Layer* is the benchmark’s landscape of 273 resources across ten enterprise systems. The *Description Layer* is their ORD v1.16 metadata, harvested into a single registry the pipeline queries. The *Semantic Layer* is instantiated by running the design-time process-matching flow over the 30 ORD-Bench process models, which writes the four typed field groups back into the resources and thereby produces the Enriched-ORD state evaluated against the clean one. The *Orchestration Layer* is the runtime pipeline itself: its Intent Resolver, Skill Registry (the 30 derived skills), Planner, and Mode Selection are exercised by the 110 runtime cases. The architecture is thus not evaluated in the abstract but as a working instantiation whose every layer is backed by a benchmark artefact; the remainder of this section reports how well that instantiation performs.

Orchestration. Reporting starts from the routing stage, before turning to the individual modes. Figure 4 gives the

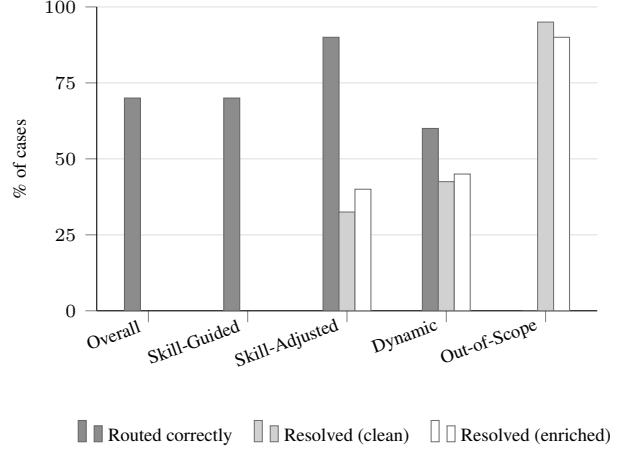


Figure 4: End-to-end view over the 110 runtime cases. *Overall* (left): combined routing accuracy (70.0%). Per-mode breakdown (right): dark grey = routing; mid-grey/white = resolution on clean/enriched states, R@1 for Skill-Adjusted (Embedding) and Dynamic (Agentic Hybrid), refusal for Out-of-Scope (Agentic Raw). Overall resolution and Skill-Guided resolution are blank (mixed metrics / no retrieval stage). Stages side by side, not multiplied.

combined end-to-end view across all 110 cases, placing the routing stage next to the subsequent retrieval stage for the best method per mode. Across the 90 routable cases (SG, SA, and DY; out-of-scope is excluded here, as it has no correct routing target), the Intent Resolver reaches a weighted average routing accuracy of 70.0% (SA: 90.0%, SG: 70.0%, DY: 60.0%). Routing and retrieval are reported as separate stages rather than multiplied into one number.

The three orchestration modes differ in how far routing alone determines the outcome. Skill-Guided is evaluated by routing accuracy alone: its step resources are pinned in the matched skill, so it bypasses the Planner and Stage 3 entirely. Skill-Adjusted requests are routed most reliably, as they provide stronger contextual signals than exact skill matches. Dynamic requests are the most challenging because distinguishing the absence of a suitable skill from a missed skill match requires reasoning over an open request. Accordingly, the Planner performs reliably when identifying gaps in an existing skill but is less successful at decomposing unstructured requests into the exact set of required sub-queries. Representative runtime traces for Skill-Guided (Appendix C.2), Skill-Adjusted (Appendix C.3), and Dynamic execution with Agentic Hybrid (Appendix C.4) illustrate the routing pipeline in Appendix B.5.

Retrieval strategies. As shown by the orchestration results, retrieval is required only for the two execution modes that discover resources at runtime: Skill-Adjusted and Dynamic. Skill-Guided executes predefined resources and therefore does not exercise the retrieval layer. On these two modes, the best-performing method resolves the correct resource at rank one (recall at rank 1, R@1) in 32.5% of Skill-Adjusted and 42.5% of Dynamic cases without enrichment, increasing to 40.0% and 45.0% with enrichment (Fig. 4). These values represent the highest accuracy achieved per exe-

Table 4: Runtime results for the two retrieval-driven modes. Skill-Guided is omitted as it performs no retrieval (routing only, 70.0% accuracy, Fig. 4), and Out-of-Scope is reported separately in Tab. 5 since it is scored by refusal rather than R@1/R@5. R@1 = recall at rank 1; R@5 = recall within top 5. Ref. = OOS refusal rate. Δ = Enriched – Clean. Bold = best per column. Values in % (R@1, R@5, Ref.), Δ in pp. Tokens = average per case.

Method	Skill-Adjusted ($n=20$)					Dynamic ($n=40$)					Tok.
	w/o enr.		w/ enr.		Δ	w/o enr.		w/ enr.		Δ	
	R@1	R@5	R@1	R@5	R@1	R@1	R@5	R@1	R@5	R@1	
Baseline	25.0	60.0	25.0	60.0	± 0.0	16.3	46.3	16.3	46.3	± 0.0	24–44k
Embedding	32.5	65.0	40.0	60.0	+7.5	32.5	67.5	28.7	70.0	−3.8	~1k
Progressive	25.0	47.5	25.0	55.0	± 0.0	13.8	28.7	13.8	27.5	± 0.0	6–11k
Graph	5.0	10.0	10.0	15.0	+5.0	2.5	7.5	12.5	17.5	+10.0	7–12k
Tools	25.0	30.0	37.5	47.5	+12.5	25.0	31.2	40.0	48.7	+15.0	75–85k
Raw	5.0	5.0	2.5	2.5	−2.5	12.5	12.5	10.0	10.0	−2.5	52–141k
Hybrid	32.5	60.0	30.0	50.0	−2.5	42.5	51.2	45.0	53.7	+2.5	23–36k

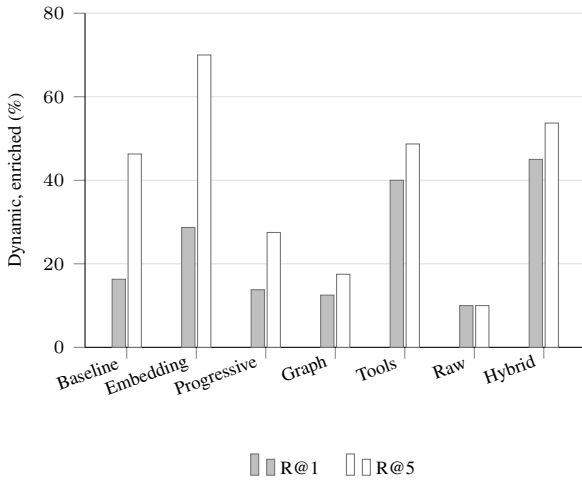


Figure 5: Rank-1 recall (R@1) versus top-5 recall (R@5) per method on Dynamic under enrichment. The gap is widest for Embedding (28.7 vs. 70.0). Typed-field methods (Agentic Tools, Agentic Hybrid) narrow the gap by improving rank-1 accuracy.

cution mode. Table 4 reports the results of all retrieval methods on both ORD description states, as R@1 and R@5 (recall within the top 5). The different retrieval strategies are discussed below.

The free-text *Baseline* establishes the reference point, reaching $R@1 \approx 16\text{--}25\%$ without an explicit retrieval mechanism. *Embedding* improves this by roughly $1.3\times$ on Skill-Adjusted and $2\times$ on Dynamic in the clean state. It also achieves the highest R@5 across all modes (up to 70.0% on Dynamic with enrichment) while requiring only $\sim 1k$ tokens per case, making it the least expensive method. Its R@1 changes inconsistently under enrichment, increasing by +7.5 pp on Skill-Adjusted but decreasing by 3.8 pp on Dynamic. *Progressive* performs hierarchical candidate filtering over the ORD structure. It requires 6–11k tokens per case and shows no change under enrichment, with R@1 remaining at 25.0% on Skill-Adjusted and 13.8% on Dynamic across both ORD states. *Graph* traverses typed ORD relations before re-ranking the results. Under enrichment, its R@1 increases by +5.0 pp on Skill-Adjusted (to 10.0%)

Table 5: Out-of-scope results ($n=20$). Δ = enriched – baseline on refusal. Embedding and Baseline were evaluated without an explicit refusal mechanism (0% refusal). Bold = best.

Method	Refusal Rate		False-Pick		Avg. Tokens		Δ
	w/o	w/	w/o	w/	w/o	w/	
Baseline	0.0	0.0	100	100	15k	15k	± 0.0
Embedding	0.0	0.0	100	100	1.2k	1.6k	± 0.0
Progressive	25.0	35.0	75	65	5.3k	5.8k	+10.0
Graph	65.0	85.0	40	15	3.8k	5.7k	+20.0
Tools	40.0	45.0	60	55	31k	40k	+5.0
Raw	95.0	90.0	5	10	52k	22k	−5.0
Hybrid	15.0	30.0	85	70	26k	25k	+15.0

and by +10.0 pp on Dynamic (to 12.5%).

The *agentic* methods expose the resource landscape through an iterative tool loop. *Agentic Tools* shows the largest improvement under enrichment, reaching $R@1 = 40.0\%$ on Dynamic (+15.0 pp). *Agentic Raw* is both the most expensive method (up to 141k tokens per case) and among the weakest performers. *Agentic Hybrid* achieves the highest Dynamic performance ($R@1 = 45.0\%$ with enrichment, more than $2.7\times$ the Baseline) but reaches 32.5% on Skill-Adjusted, matching the best individual method.

Across all methods, R@5 consistently exceeds R@1: the correct resource is present in the candidate set far more often than it is ranked first, so the bottleneck is ranking rather than coverage. Figure 5 visualises this gap across all methods. A breakdown of the Dynamic cases into single- and multi-intent requests is provided in Appendix E.

Out-of-scope. Out-of-scope cases are scored separately from the retrieval modes above, because success here is the *absence* of a pick rather than a rank: the correct behaviour is refusal, so R@1/R@5 do not apply and refusal rate is reported instead (Tab. 5). Graph and the agentic methods carry an explicit `refuse` action. Agentic Raw refuses most reliably (95.0% without enrichment, −5.0 pp under it) but at the highest cost; Graph is the most cost-efficient, refusing 65.0% without enrichment and 85.0% with it (+20.0 pp) at just 3.7–5.7k tokens; Agentic Tools refuses steadily at

	SA	DY	OOS
Baseline	± 0.0	± 0.0	± 0.0
Embedding	+7.5	-3.8	± 0.0
Progressive	± 0.0	± 0.0	+10.0
Graph	+5.0	+10.0	+20.0
Tools	+12.5	+15.0	+5.0
Raw	-2.5	-2.5	-5.0
Hybrid	-2.5	+2.5	+15.0

Green = gain Red = degradation

Figure 6: Enrichment effect Δ (pp) per method and mode. Agentic Tools gains most in SA and DY (+12.5, +15.0 pp); Graph gains most in OOS (+20.0 pp); Embedding degrades on DY (-3.8 pp).

40–45% and Agentic Hybrid at 15–30%. Appendix C.5 provides a representative example trace using Graph Walk.

Embedding and Baseline provide no rejection mechanism by design and therefore always return a top-ranked resource, leading to false selections on all 20 out-of-scope cases. Confidence-based rejection was evaluated only for methods that natively expose such a mechanism. For Embedding, selecting and validating a similarity threshold is left to future work given the limited sample size, while the Baseline is included as a non-retrieval reference. Possible rejection mechanisms are discussed in Sec. 5.

Semantic enrichment. Fig. 6 visualises the enrichment effect $\Delta = \text{Enriched} - \text{Clean}$ across all methods and modes. The effect divides along how a method consumes the enriched fields. Methods that read them as typed signals improve most: Agentic Tools gains +12.5 pp on Skill-Adjusted and +15.0 pp on Dynamic, and Graph gains +5.0 to +10.0 pp on retrieval and +20.0 pp on out-of-scope refusal, its largest single gain. Embedding, which folds the same fields into its text vector rather than reading them as discrete signals, moves in both directions: it gains +7.5 pp on Skill-Adjusted but loses -3.8 pp on Dynamic. Progressive is largely unaffected on retrieval (± 0.0 pp on both modes), as its early filtering stages read structure that is identical in the two states, though its refusal does rise (+10.0 pp on out-of-scope). Agentic Raw degrades throughout (-2.5 pp on retrieval, -5.0 pp on refusal), the added capability text giving its unconstrained file reading more plausible-looking but irrelevant material. Semantic enrichment is therefore not uniformly beneficial; its sign depends on the method and the mode.

Statistical reliability. With only 20 to 40 cases per execution mode, observed performance differences may reflect sampling variability rather than true differences between retrieval methods. Two complementary analyses therefore assess whether the paper’s overarching findings, rather than the exact per-method scores, hold up under this uncertainty. The first analysis quantifies the uncertainty of each reported score. Resampling the per-case outcomes 10,000 times (*bootstrap*) yields a 95% confidence interval, the range in

which the true performance is likely to lie. On the two strongest methods on Dynamic (enriched, $n=40$), Agentic Hybrid reaches $R@1 = 45.0\%$ ([31.2, 58.8]) and Embedding $R@5 = 70.0\%$ ([57.5, 82.5]) but only $R@1 = 28.7\%$ ([16.2, 42.5]). The intervals are wide, so individual scores should not be read to the percentage point, though one qualitative pattern is already visible: no method is best on every metric, as Embedding leads on recall ($R@5$) and Agentic Hybrid on rank-one precision ($R@1$). The choice of method thus depends on whether the application needs a broad candidate set or a precise top-one pick.

The second analysis tests two distinct claims that are easily conflated. The first, that *typed retrieval structure* beats free text, is confirmed by a paired McNemar test on the same cases ($p < 0.05$ meaning the difference is unlikely to be chance): Agentic Hybrid ($p = 0.008$) and Agentic Tools ($p = 0.019$) significantly outperform the free-text Baseline. The second and stronger claim, that *semantic enrichment itself* improves selection, requires comparing each method against its own clean-state score. The relevant quantity is the enrichment delta (enriched minus clean $R@1$), where zero means enrichment changed nothing; a bootstrap interval that stays entirely above zero therefore signals a genuine gain rather than noise. This interval excludes zero for only two methods, Agentic Tools (+15.0 pp, [+2.5, +27.5]) and Graph (+10.0 pp, [+2.5, +20.0]), precisely the typed-field methods that consume the enriched fields; for the rest the interval spans zero, so no reliable effect can be claimed at this sample size. The evaluation therefore supports two qualitative conclusions, that typed structure beats free text and that enrichment helps the methods aligned to consume it, but not a precise ordering of the strongest methods, which stays within noise ($p > 0.15$).

5. Discussion

This section first interprets the results with respect to the research question, then examines each contribution in turn (the architecture, the retrieval strategies, the enrichment effect, and the benchmark), before discussing the limitations of the work and directions for future research.

Interpretation of findings. The results show *how* the research question is answered: resource selection is enabled by routing each request to one of three orchestration modes according to its process structure, and by resolving it against a typed ORD description layer rather than free text. Semantic enrichment strengthens selection further, but conditionally: it helps only the retrieval mechanisms whose architecture consumes typed fields, and leaves free-text methods flat or degrades them. The practical implication is that enrichment is not a universal lever. Enterprises should audit their retrieval architecture before investing in metadata enrichment.

Process-aware architecture. The central contribution is the four-layer separation itself: decoupling the resource landscape, its ORD description layer, the process-derived semantic layer, and the orchestration logic is what lets a single pipeline select resources across the full process-

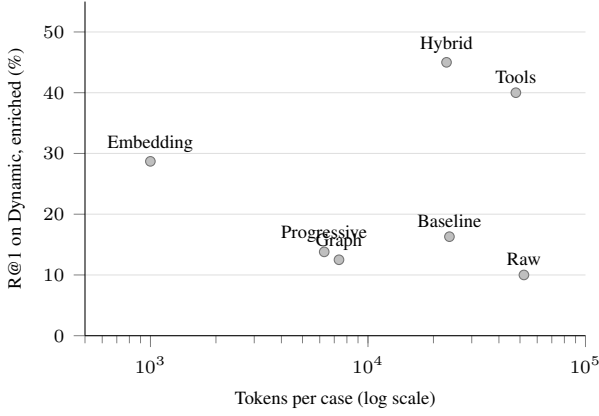


Figure 7: Cost-precision trade-off on Dynamic under enrichment: average tokens per case (log scale) versus R@1. Embedding is near-free (~1k tokens) at 28.7% R@1; Agentic Hybrid reaches the highest precision (45.0%) at moderate cost; Agentic Raw is the most expensive yet weakest.

structuredness spectrum. Each layer is exercised by the 110 benchmark cases and the ten requirements map cleanly onto the four layers (Appendix B.6), so the design is demonstrated end to end rather than only argued for. Its value over prior work is that A-BPMS proposals [2, 50] defined orchestration layers in increasing detail but left the resource-selection mechanism unspecified, whereas the layered design here supplies exactly that mechanism, and it realises the long-standing SBPM vision of machine-readable process-to-service links [37] without the ontology overhead that limited its adoption, using LLM reasoning over typed ORD fields in place of hand-authored ontologies.

Practitioners confirm both the foundation and the pipeline shape. The architect responsible for the ORD specification notes that its productive use at SAP implicitly confirms the value of a machine-readable self-description layer, and that a unified, consistently-published metadata standard is the prerequisite for any orchestration layer built on top (I2, Appendix A.2). This productive use underlines the practical value for enterprises of establishing a unified description layer over their resources that can be semantically extended.

Within this architecture, the orchestration layer’s routing stage is the clearest gap to production readiness, though the per-mode split is informative. Skill-Adjusted routes most reliably because partially-matching requests carry stronger disambiguation signals, whereas Dynamic is hardest because a genuine “no skill” case is difficult to separate from a missed match. Two properties of the layered design soften the impact of routing errors. A Skill-Guided miss falls through to Skill-Adjusted where the Planner compensates, and a Dynamic miss only inflates token cost without affecting which resources are reachable. The design implication is that the structural property of a request should determine the orchestration mode rather than a static routing rule. In production, a confidence threshold that escalates ambiguous requests to the more expensive mode by default, complemented by UX-side clarification when confidence is low, would resolve most remaining routing errors.

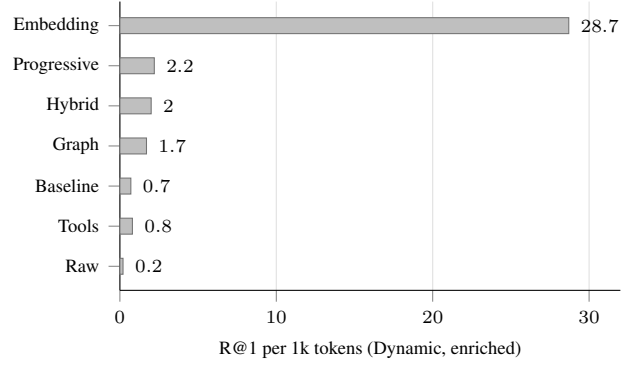


Figure 8: Token efficiency: R@1 per 1,000 tokens on Dynamic under enrichment. Embedding is more than an order of magnitude more cost-efficient than any other method, since it reaches competitive precision at ~1k tokens. The agentic methods (Agentic Tools, Agentic Raw) buy precision at a steep token cost, and the Agentic Hybrid composite sits in between.

Retrieval. No single method dominates and all absolute accuracies stay below production requirements, which follows from ORD-Bench’s adversarial construction, where every case is surrounded by semantically close distractors and trivially solvable cases are filtered out.

The consistently high R@5 relative to R@1 (Fig. 5) re-frames the optimisation target. Recall is already near its ceiling, so the lever is converting existing R@5 into R@1 through a lightweight re-ranking step over the top-five rather than a different retrieval architecture. This is exactly the two-stage shape that two practitioners independently describe, an embedding-based pre-selection narrowed by an LLM decision over the shortlist (I3, I4, Appendix A.2).

Still, accuracy alone does not decide method choice, since cost varies by more than an order of magnitude across methods. Two views make this concrete. The cost-precision trade-off (Fig. 7) shows that no method wins on both axes: Embedding is near-free, since it needs no per-query LLM call and only a single query embedding, but scores low on R@1, while the agentic methods buy higher R@1 at a steep token cost through repeated LLM calls. Only Embedding, Agentic Hybrid, and Agentic Tools are not beaten by another method on both cost and R@1 at once, and so represent the sensible choices; the rest are dominated. Token efficiency (Fig. 8) sharpens the same picture into a single measure of R@1 per thousand tokens, on which Embedding is more than an order of magnitude ahead of any other method, the agentic methods sit at the opposite end, and Agentic Hybrid falls in between.

Together these views make method selection a deployment trade-off rather than a search for a single winner. A cost-sensitive or high-throughput deployment should default to embedding retrieval and reserve the agentic methods for the cases where their precision gain justifies the cost, a boundary that the growing user tolerance for slower, more deliberate assistants is steadily shifting in favour of the agentic methods (I3, Appendix A.2).

Failure modes. To characterise why methods fail and de-

Table 6: Failure-mode breakdown on Dynamic (enriched), as % of the *failed* cases per method. Rank err. = target in top-5 but not first; Retr. miss = target not retrieved; Empty = committed to nothing. Rows sum to 100% of each method’s failures.

Method	Rank err.	Retr. miss	Empty
Baseline	57	43	0
Embedding	68	32	0
Progressive	28	70	2
Graph	22	65	13
Tools	50	50	0
Raw	2	60	38
Hybrid	32	68	0

rive concrete directions for improving them, every Dynamic case (enriched) was classified by failure type from the per-case traces (Tab. 6). A *rank error* means the correct resource was retrieved into the top five but not ranked first; a *retrieval miss* means it was absent from the candidate set entirely; an *empty return* means the method committed to nothing despite a valid target.

The value of this breakdown is that each dominant failure mode implies a concrete next step for the affected method rather than a generic call for higher accuracy. Methods dominated by rank errors, need a re-ranking stage over the shortlist rather than better recall, since the target is already present. Methods dominated by retrieval misses, such as the filter-based ones, need their hard early filters softened into soft scores that never prune a namespace to zero. Methods with high empty-return rates need their abstention threshold relaxed or their raw file access replaced with typed tools. The failure profile thus turns each method’s aggregate score into an actionable direction for its further development.

Semantic enrichment. The most consequential finding is that enrichment is not a universal good. Frameworks such as MCP [7] and A2A [8] implicitly assume that attaching richer natural-language metadata to a resource makes it easier to select. The evaluation shows the opposite can hold, as the same enriched fields that lift the typed-field methods leave free-text methods flat and actively degrade Embedding. What decides the sign of the effect is not how much metadata is added but whether the retrieval mechanism can address it as structure. A capability filter or a graph edge turns a populated field into a discriminative signal, whereas a single embedding vector folds it back into undifferentiated text and can pull a resource closer to its distractors. Enrichment and retrieval are therefore not independent levers but one coupled design decision.

This reframes the practical question for enterprises from whether to enrich their resource descriptions to whether their retrieval stack consumes typed structure at all. Investing in metadata enrichment without a typed-field retrieval mechanism yields no benefit and may reduce accuracy, so the retrieval architecture should be decided before the enrichment effort, not after. A knowledge-graph practitioner reports the same dependency from production, noting that

reliable disambiguation needs use-case- or process-based search over a typed schema rather than plain embeddings, which “become noisy when multiple resources have similar scope” (15, Appendix A.2). The ORD-Bench paper grounds the mechanism structurally, finding that the identical enrichment lowers structural ambiguity by -24.5% yet raises embedding cosine by $+27.7\%$ on the same resource pairs, the opposite signs being the structural signature of the mechanism-dependence observed here.

Benchmark. The results hold only under the conditions ORD-Bench enforces. Because every case is adversarially constructed, with the correct resource surrounded by semantically close distractors and trivial cases filtered out, absolute accuracies are lower than an uncurated catalogue would produce, and the ranking between methods is meaningful precisely for that reason. Using ORD-Bench was necessary, since no other benchmark evaluates retrieval over ORD or ships typed enterprise resources in paired clean and enriched states over one fixed landscape. That pairing is what lets this paper isolate the per-method enrichment effect instead of confounding it with a change in the resource set. The study inherits ORD-Bench’s assumptions of a synthetic, mid-scale, single-vendor-vocabulary landscape, so the absolute numbers are specific to it while the directional findings are expected to transfer. ORD-Bench is the subject of a dedicated companion paper, which covers its construction and its positioning against prior benchmarks; these are not revisited here.

Limitations. A first group of limitations concerns the preconditions and the method setup. The architecture rests on three preconditions: a unified entity-type vocabulary across system boundaries (e.g., SAP One Domain Model), sufficient ORD metadata coverage, and process models to drive the Skill-Guided and Skill-Adjusted modes. The retrieval methods are deliberately kept untuned, each instantiating one family with a fixed prompt, so that measured differences are attributable to the description layer rather than to optimisation. The absolute numbers are therefore conservative baselines, and more capable implementations of any family (fine-tuned embeddings, calibrated thresholds, learned tool-selection policies) can be expected to raise precision while preserving the enrichment-conditionality finding. All LLM calls used Claude Haiku 4.5; absolute accuracies may shift under stronger models.

A second group concerns evaluation scope and statistical power. The study stops at resource selection and does not measure whether the selected resource executes correctly in a process. ORD-Bench uses LLM-generated resources and single-annotator ground truth without inter-annotator agreement or a held-out partition. At $n=20-40$ per split, each 5.0 pp step is one or two cases, so the intervals are wide and most pairwise differences are not statistically separable (Sec. 4). The magnitudes are specific to this landscape while the directional findings should generalise, and larger case counts are the clearest path to decisive results.

A final group concerns the cost of the approach. The semantic ORD extension carries an ongoing burden, since

sustaining enriched fields needs maintenance and ownership rules and the four field groups are partly user-defined extensions rather than standardised ORD core.

Future work. On the evaluation side, running more test cases over a larger landscape, which requires scaling ORD-Bench itself, would narrow the wide confidence intervals and make the pairwise method differences statistically decisive. Independent of benchmark scale, the pipeline can be advanced directly: replacing the single-call Intent Resolver with a retrieval-augmented variant, validating the results under a frontier model, and extending the evaluation from resource selection to end-to-end task completion on harnesses such as τ -bench [54]. The retrieval methods themselves can be pushed further, for instance through learned re-ranking over the top-five shortlist where recall is already high but ranking is not.

On the enrichment side, a dropout-style ablation that removes one field group at a time would isolate which fields carry the disambiguating signal, and a complementary *enriched-only* ablation, exposing the four process-derived fields with the base free-text removed, would isolate their standalone discriminative power and clarify whether the observed embedding degradation stems from mixing typed signal into base text rather than from semantics themselves.

A further direction is also a *self-optimising* orchestrator and retriever that improves during operation rather than relying on fixed logic, a path interviewed practitioners see as the only way to scale across thousands of customised scenarios (I1, I5, Appendix A.2). Rather than a manually curated intent-resource dataset, such a system could learn from usage signals, explicit feedback or successful downstream execution as rewards. Agentic Hybrid already provides the action space for this, and contextual-bandit formulations that treat each retrieval strategy as an arm offer an established grounding [55, 56]. The same signals could also refine resource representations through parameter-efficient adapters over a frozen encoder, without altering the descriptions or the deterministic typed-ORD routing [57, 58].

6. Conclusion

This work set out to answer the question: *How can a process-aware orchestration architecture and semantic resource descriptions enable resource selection by LLM-based agents in heterogeneous enterprise landscapes?* It addressed this through three contributions, each targeting one of the coordination challenges identified at the outset. (1) A *process-aware reference architecture* layers a unified ORD description layer and a process-derived semantic layer beneath an agentic orchestration layer with three modes, spanning the full process-structuredness spectrum and thereby supporting coexistence with established processes. (2) *Seven retrieval strategies* over the typed ORD layer make the trade-off between cost, selectivity, and refusal observable, addressing action-space complexity. (3) A *systematic comparison* of these strategies with and without semantic enrichment, on 110 ORD-Bench cases.

Together these contributions answer the research question: the process-aware architecture enables reliable resource selection across the full process spectrum, and semantic grounding improves it further, but only for methods whose architecture consumes the typed fields. Structural methods gain up to +15.0 pp under enrichment while free-text methods stay flat or degrade, and the consistently high R@5 shows the architecture reliably surfaces the correct resource in the top five, leaving ranking it first as the remaining challenge. The core lesson is that enrichment and retrieval are one coupled decision: richer descriptions help only where the retrieval mechanism can consume them as structure.

Beyond the technical next steps, the broader trajectory points toward orchestrators executing entire business processes through natural-language intent alone. For this vision, a unified, LLM-accessible description and semantic layer must be in place across all enterprise systems, and this work provides the empirical foundation for that trajectory.

References

- [1] W. Xu, C. Huang, S. Gao, and S. Shang, “Llm-based agents for tool learning: A survey,” *Data Sci. Eng.*, vol. 10, no. 4, pp. 533–563, 2025. [Online]. Available: <https://doi.org/10.1007/s41019-025-00296-9>
- [2] M. Dumas, F. Milani, and D. Chapela-Campa, “Agentic business process management systems,” in *Business Process Management Workshops - BPM 2025 International Workshops, Seville, Spain, August 31-September 5, 2025, Revised Selected Papers*, ser. Lecture Notes in Business Information Processing, I. van de Weerd, B. Estrada-Torres, and H. van der Aa, Eds., vol. 569. Springer, 2025, pp. 3–14. [Online]. Available: https://doi.org/10.1007/978-3-032-13426-4_1
- [3] Y. Tang, W. Su, Y. Liu, and Q. Ai, “Multi-field tool retrieval,” *CoRR*, vol. abs/2602.05366, 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2602.05366>
- [4] X. Lu, H. Huang, R. Meng, Y. Jin, W. Zeng, and X. Shen, “Tools are under-documented: Simple document expansion boosts tool retrieval,” in *The Fourteenth International Conference on Learning Representations* 26. [Online]. Available: <https://openreview.net/forum?id=g9D9MgG7iW>
- [5] W. M. P. van der Aalst, “Business process management: A comprehensive survey,” *ISRN Software Engineering*, vol. 2013, pp. 1–37, 2013. [Online]. Available: <https://doi.org/10.1155/2013/507984>
- [6] C. D. Ciccio, A. Marrella, and A. Russo, “Knowledge-intensive processes: Characteristics, requirements and analysis of contemporary approaches,” *J. Data Semant.*, vol. 4, no. 1, pp. 29–57, 2015. [Online]. Available: <https://doi.org/10.1007/s13740-014-0038-4>
- [7] Specification. [Online]. Available: <https://modelcontextprotocol.io/specification/2025-11-25>

- [8] A. A. Catalog. AI agent marketplace - discover a2a agents, MCP servers, and agentic workflows. [Online]. Available: <https://a2acatalog.com>
- [9] M. M. Hasan, H. Li, G. K. Rajbahadur, B. Adams, and A. E. Hassan, "Model context protocol (MCP) tool descriptions are smelly! towards improving AI agent efficiency with augmented MCP tool descriptions." [Online]. Available: <https://arxiv.org/abs/2602.14878>
- [10] S. A. McIlraith, T. C. Son, and H. Zeng, "Semantic web services," *IEEE Intell. Syst.*, vol. 16, no. 2, pp. 46–53, 2001. [Online]. Available: <https://doi.org/10.1109/5254.920599>
- [11] H. Österle, J. Becker, U. Frank, T. Hess, D. Karagianis, H. Krcmar, P. Loos, P. Mertens, A. Oberweis, and E. J. Sinz, "Memorandum on design-oriented information systems research," *Eur. J. Inf. Syst.*, vol. 20, no. 1, pp. 7–10, 2011. [Online]. Available: <https://doi.org/10.1057/ejis.2010.55>
- [12] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach (4th Edition)*. Pearson, 2020. [Online]. Available: <http://aima.cs.berkeley.edu/>
- [13] S. Franklin and A. Graesser, "Is it an agent, or just a program?: A taxonomy for autonomous agents," in *Intelligent Agents III Agent Theories, Architectures, and Languages*. Springer, pp. 21–35.
- [14] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., 2023. [Online]. Available: http://papers.nips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html
- [15] OpenAI. Function calling and other API updates. [Online]. Available: <https://openai.com/index/function-calling-and-other-api-updates/>
- [16] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive apis," in *Advances in Neural Information Processing Systems 37: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, A. Globersons, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. M. Tomczak, and C. Zhang, Eds., 2024. [Online]. Available: http://papers.nips.cc/paper_files/paper/2024/hash/e4c61f578ff07830f5c37378dd3ecb0d-Abstract-Conference.html
- [17] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., 2020. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>
- [18] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023. [Online]. Available: https://openreview.net/forum?id=WE_vluYUL-X
- [19] Z. Shi, Y. Wang, L. Yan, P. Ren, S. Wang, D. Yin, and Z. Ren, "Retrieval models aren't tool-savvy: Benchmarking tool retrieval for large language models," in *Findings of the Association for Computational Linguistics, ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, ser. Findings of ACL, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds., vol. ACL 2025. Association for Computational Linguistics, 2025, pp. 24 497–24 524. [Online]. Available: <https://doi.org/10.18653/v1/2025.findings-acl.1258>
- [20] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, "Toolllm: Facilitating large language models to master 16000+ real-world apis," in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. [Online]. Available: <https://openreview.net/forum?id=dHng2O0Jjr>
- [21] F. Nizar, E. Lumer, A. Gulati, P. H. Basavaraju, and V. K. Subbiah, "Agent-as-a-graph: Knowledge graph-based tool and agent retrieval for LLM multi-agent systems," in *Proceedings of the 18th International Conference on Agents and Artificial Intelligence, ICAART 2026 - Volume 3, Marbella, Spain, March 5-7, 2026*, A. P. Rocha, M. Wahde, and H. J. van den Herik, Eds. SCITEPRESS, 2026, pp. 1975–1984. [Online]. Available: <https://doi.org/10.5220/0014473600004052>
- [22] C. Qu, S. Dai, X. Wei, H. Cai, S. Wang, D. Yin, J. Xu, and J. Wen, "Towards completeness-oriented tool retrieval for large language models," in *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, CIKM*

- 2024, Boise, ID, USA, October 21-25, 2024. ACM, 2024, pp. 1930–1940. [Online]. Available: <https://doi.org/10.1145/3627673.3679847>
- [23] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, and J. Larson, “From local to global: A graph RAG approach to query-focused summarization,” April 2024.
- [24] I. Melzer, *Service-Orientierte Architekturen Mit Web Services: Konzepte - Standards - Praxis*, 4th ed. Spektrum Akademischer Verlag GmbH.
- [25] F. Curbera, M. J. Duftler, R. Khalaf, W. Nagy, N. Mukhi, and S. Weerawarana, “Unraveling the web services web: An introduction to soap, wsdl, and UDDI,” *IEEE Internet Comput.*, vol. 6, no. 2, pp. 86–93, 2002. [Online]. Available: <https://doi.org/10.1109/4236.991449>
- [26] S. Schwichtenberg, C. Gerth, and G. Engels, “From open API to semantic specifications and code adapters,” in *2017 IEEE International Conference on Web Services, ICWS 2017, Honolulu, HI, USA, June 25-30, 2017*, I. Altintas and S. Chen, Eds. IEEE, 2017, pp. 484–491. [Online]. Available: <https://doi.org/10.1109/ICWS.2017.56>
- [27] C. Pautasso, O. Zimmermann, and F. Leymann, “Restful web services vs. “big” web services: making the right architectural decision,” in *Proceedings of the 17th International Conference on World Wide Web, WWW 2008, Beijing, China, April 21-25, 2008*, J. Huai, R. Chen, H. Hon, Y. Liu, W. Ma, A. Tomkins, and X. Zhang, Eds. ACM, 2008, pp. 805–814. [Online]. Available: <https://doi.org/10.1145/1367497.1367606>
- [28] N. Dragoni, S. Giallorenzo, A. Lluch-Lafuente, M. Mazzara, F. Montesi, R. Mustafin, and L. Safina, “Microservices: Yesterday, today, and tomorrow,” in *Present and Ulterior Software Engineering*, M. Mazzara and B. Meyer, Eds. Springer, 2017, pp. 195–216. [Online]. Available: https://doi.org/10.1007/978-3-319-67425-4_12
- [29] The Open Semantic Interchange authors. Open semantic interchange. [Online]. Available: <https://open-semantic-interchange.org/>
- [30] J. Bu and S. Krishnan. Announcing the agentic resource discovery specification-google developers blog. [Online]. Available: <https://developers.googleblog.com/announcing-the-agentic-resource-discovery-specification/>
- [31] Linux Foundation Europe. Open resource discovery. [Online]. Available: <https://open-resource-discovery.org/>
- [32] SAP SE. Apeiro reference architecture. [Online]. Available: <https://apeirora.eu/>
- [33] F. Schmiedchen, A. von Gernler, M. Hafner, and K. P. Kratzer, Eds., *Künstliche Intelligenz und Wir: Stand, Nutzung und Herausforderungen der KI*. Springer Berlin Heidelberg, 2026. [Online]. Available: <https://doi.org/10.1007/978-3-662-71567-3>
- [34] A. Hogan, E. Blomqvist, M. Cochez, C. d’Amato, G. d. Melo, C. Gutierrez, S. Kirrane, J. E. L. Gayo, R. Navigli, S. Neumaier *et al.*, “Knowledge graphs,” *ACM Computing Surveys*, vol. 54, no. 4, pp. 1–37, 2021. [Online]. Available: <https://arxiv.org/abs/2003.02320>
- [35] T. Berners-Lee and J. Hendler, “Publishing on the semantic web,” *Nature*, vol. 410, no. 6832, pp. 1023–1024, 2001. [Online]. Available: <https://doi.org/10.1038/35074206>
- [36] M. Hepp, F. Leymann, J. Domingue, A. Wahler, and D. Fensel, “Semantic business process management: A vision towards using semantic web services for business process management,” pp. 535–540, 2005.
- [37] C. Pedrinaci, J. Domingue, C. Brelage, T. van Lessen, D. Karastoyanova, and F. Leymann, “Semantic business process management: Scaling up the management of business processes,” in *Proceedings of the 2th IEEE International Conference on Semantic Computing (ICSC 2008), August 4-7, 2008, Santa Clara, California, USA*. IEEE Computer Society, 2008, pp. 546–553. [Online]. Available: <https://doi.org/10.1109/ICSC.2008.84>
- [38] C. Feier, A. Polleres, D. Roman, J. Domingue, M. Stollberg, and D. Fensel, “Towards intelligent web services: The web service modeling ontology (WSMO).” [Online]. Available: <https://oro.open.ac.uk/23147/1/10.1.1.94.1336%5B1%5D.pdf>
- [39] S. Balzer, T. Liebig, and M. Wagner, “Pitfalls of OWL-S: a practical semantic web use case,” in *Service-Oriented Computing - ICSOC 2004, Second International Conference, New York, NY, USA, November 15-19, 2004, Proceedings*, M. Aiello, M. Aoyama, F. Curbera, and M. P. Papazoglou, Eds. ACM, 2004, pp. 289–298. [Online]. Available: <https://doi.org/10.1145/1035167.1035209>
- [40] M. Hepp and D. Roman, “An ontology framework for semantic business process management,” in *eOrganisation: Service-, Prozess-, Market-Engineering: 8. Internationale Tagung Wirtschaftsinformatik - Band I, WI 2007, Karlsruhe, Germany, February 28 - March 2, 2007*, A. Oberweis, C. Weinhardt, H. Gimpel, A. Koschmider, V. Pankratius, and B. Schnizler, Eds. Universitaetsverlag Karlsruhe, 2007, pp. 423–440. [Online]. Available: <http://aisel.aisnet.org/wi2007/27>
- [41] R. Wang, X. Han, L. Ji, S. Wang, T. Baldwin, and H. Li, “ToolGen: Unified tool retrieval and calling via generation,” in *The Thirteenth International*

- Conference on Learning Representations (ICLR)*. [Online]. Available: <https://openreview.net/forum?id=XLMMmowdY>
- [42] Equipping agents for the real world with agent skills. [Online]. Available: <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>
- [43] G. Aagesen and J. Krogstie, “BPMN 2.0 for modeling business processes,” in *Handbook on Business Process Management 1, Introduction, Methods, and Information Systems, 2nd Ed*, ser. International Handbooks on Information Systems, J. vom Brocke and M. Rosemann, Eds. Springer, 2015, pp. 219–250. [Online]. Available: https://doi.org/10.1007/978-3-642-45100-3_10
- [44] Object Management Group, “Case management model and notation (CMMN) version 1.1,” 2016. [Online]. Available: <https://www.omg.org/spec/CMMN/1.1/PDF>
- [45] H. R. Motahari-Nezhad and K. D. Swenson, “Adaptive case management: Overview and research challenges,” in *2013 IEEE 15th Conference on Business Informatics*, 2013, pp. 264–269. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/6642886>
- [46] M. Rosemann, J. vom Brocke, A. V. Looy, and F. M. Santoro, “Business process management in the age of AI - three essential drifts,” *Inf. Syst. E Bus. Manag.*, vol. 22, no. 3, pp. 415–429, 2024. [Online]. Available: <https://doi.org/10.1007/s10257-024-00689-9>
- [47] M. Vidgof, S. Bachhofner, and J. Mendling, “Large language models for business process management: Opportunities and challenges,” in *Business Process Management Forum - BPM 2023 Forum, Utrecht, The Netherlands, September 11-15, 2023, Proceedings*, ser. Lecture Notes in Business Information Processing, vol. 490. Springer, 2023, pp. 107–123. [Online]. Available: https://doi.org/10.1007/978-3-031-41623-1_7
- [48] P. Fettke and C. D. Francescomarino, “Business process management and artificial intelligence: Literature survey and future research,” *Künstliche Intell.*, vol. 39, no. 2, pp. 67–79, 2025. [Online]. Available: <https://doi.org/10.1007/s13218-025-00891-y>
- [49] M. Vidgof, S. Bachhofner, and J. Mendling, “Large language models for business process management: Opportunities and challenges,” in *Business Process Management Forum - BPM 2023 Forum, Utrecht, The Netherlands, September 11-15, 2023, Proceedings*, ser. Lecture Notes in Business Information Processing, C. D. Francescomarino, A. Burattin, C. Janiesch, and S. Sadiq, Eds., vol. 490. Springer, 2023, pp. 107–123. [Online]. Available: https://doi.org/10.1007/978-3-031-41623-1_7
- [50] A. Adimulam, R. Gupta, and S. Kumar, “The orchestration of multi-agent systems: Architectures, protocols, and enterprise adoption,” *CoRR*, vol. abs/2601.13671, 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2601.13671>
- [51] G. Ling, S. Zhong, and R. Huang, “Agent skills: A data-driven analysis of claude skills for extending large language model functionality,” *CoRR*, vol. abs/2602.08004, 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2602.08004>
- [52] X. Li, W. Chen, Y. Liu, S. Zheng, X. Chen, Y. He, Y. Li, B. You, H. Shen, J. Sun, S. Wang, Q. Zeng, D. Wang, X. Zhao, Y. Wang, R. B. Chaim, Z. Di, Y. Gao, J. He, Y. He, L. Jing, L. Kong, X. Lan, J. Li, S. Li, Y. Li, Y. Lin, X. Liu, X. Liu, H. Lyu, Z. Ma, B. Wang, R. Wang, T. Wang, W. Ye, Y. Zhang, H. Xing, Y. Xue, S. Dillmann, and H. Lee, “Skillsbench: Benchmarking how well agent skills work across diverse tasks,” *CoRR*, vol. abs/2602.12670, 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2602.12670>
- [53] P. Mayring, *Qualitative Content Analysis: Theoretical Foundation, Basic Procedures and Software Solution*, Klagenfurt, 2014. [Online]. Available: <https://nbn-resolving.org/urn:nbn:de:0168-ssoar-395173>
- [54] S. Yao, N. Shinn, P. Razavi, and K. R. Narasimhan, “ $\{\tau\}$ -bench: A benchmark for Tool-Agent-User interaction in real-world domains,” in *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. [Online]. Available: <https://openreview.net/forum?id=roNSXZpUDN>
- [55] P. Xu, Z. Wen, H. Zhao, and Q. Gu, “Neural contextual bandits with deep representation and shallow exploration,” in *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022. [Online]. Available: <https://openreview.net/forum?id=xnYACQquaGV>
- [56] X. Tang, Q. Gao, J. Li, N. Du, Q. Li, and S. Xie, “MBA-RAG: a bandit approach for adaptive retrieval-augmented generation through question complexity,” in *Proceedings of the 31st International Conference on Computational Linguistics, COLING 2025, Abu Dhabi, UAE, January 19-24, 2025*, O. Rambow, L. Wanner, M. Apidianaki, H. Al-Khalifa, B. D. Eugenio, and S. Schockaert, Eds. Association for Computational Linguistics, 2025, pp. 3248–3254. [Online]. Available: <https://aclanthology.org/2025.coling-main.218/>
- [57] T. Schopf, D. N. Schneider, and F. Matthes, “Efficient domain adaptation of sentence embeddings using adapters,” in *Proceedings of the 14th International Conference on Recent Advances in*

Natural Language Processing, RANLP 2023, Varna, Bulgaria, 4-6 September 2023, R. Mitkov and G. Angelova, Eds. INCOMA Ltd., Shoumen, Bulgaria, 2023, pp. 1046–1053. [Online]. Available: <https://aclanthology.org/2023.ranlp-1.112>

- [58] T. Gao, X. Yao, and D. Chen, “Simcse: Simple contrastive learning of sentence embeddings,” in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*, M. Moens, X. Huang, L. Specia, and S. W. Yih, Eds. Association for Computational Linguistics, 2021, pp. 6894–6910. [Online]. Available: <https://doi.org/10.18653/v1/2021.emnlp-main.552>
- [59] Harness design for long-running application development. [Online]. Available: <https://www.anthropic.com/engineering/harness-design-long-running-apps>
- [60] D. Fensel, F. M. Facca, E. Simperl, and I. Toma, *Semantic Web Services*. Springer, 2011. [Online]. Available: <https://doi.org/10.1007/978-3-642-19193-0>
- [61] Overview - a2a protocol. [Online]. Available: <https://a2a-protocol.org/latest/specification/>

Table 7: Literature-derived requirements R1-L to R10-L, grouped under the four design objectives.

ID	Requirement	Sources
DO1 — Process Execution		
R1-L	Full process spectrum: structured, repeatable to unmodelled, knowledge-intensive.	[5, 6, 43, 44]
R2-L	Procedural knowledge as modular, dynamically loadable skills.	[42, 59]
R3-L	Agentic execution supports the implementation phase of the BPM lifecycle.	[2, 48]
DO2 — Description Layer		
R4-L	Resource discovery through a registry with typed structure.	[24, 31, 60]
R5-L	Vendor-neutral interfaces enabling interoperability.	[7, 24, 61]
DO3 — Semantic Enrichment		
R6-L	Machine-interpretable semantics with business meaning.	[10, 31, 38, 39]
R7-L	Lightweight, harvestable description layer without ontology authoring.	[31, 32, 37, 40]
R8-L	Linkage between process model artefacts and IT resources.	[37]
DO4 — Orchestration Layer		
R9-L	Disambiguate dense action spaces with overlapping descriptions.	[1, 16, 19–21]
R10-L	Robustness against “description smells” at the Semantic Layer.	[9]

A. Analysis Artefacts

This appendix documents how the requirements that structure the architecture were derived and validated. It proceeds from the literature-derived requirement catalogue, through the five expert interviews and their qualitative coding, to the consolidated final catalogue that the design chapter builds on.

A.1 Literature-Derived Requirements

This appendix documents the comprehensive extraction of requirements R1-L to R10-L from the related-work synthesis (Tab. 7). The catalogue is the deductive starting point for the expert interviews and is built before they are conducted. Each interview statement is later classified against one or more R-L entries as Confirm, Refine, or New (Appendix A.3). The ten requirements are organised under four design objectives DO1–DO4. Each entry records the supporting related-work sources.

A.2 Expert Interviews

Interview guide. The standardised interview guide corresponds directly to the questions answered by the interviewees in the transcripts below. The guide is built around the literature-derived requirements R1-L to R10-L (Appendix A.1): each question targets one or more anchor categories so that responses map back to them during coding, while open-ended questions give interviewees room to raise themes the literature did not cover.

Transcripts. The transcripts of the five expert interviews described are reproduced below. Each transcript is preceded by a short profile of the interviewee (role, area, organisational unit) and the date of the interview. Names are anonymised. The transcripts have been lightly edited for readability without changing the content.

Participants and selection. The five interviewees were selected by purposive sampling to cover the roles that jointly shape enterprise agent orchestration: product management, resource-description standardisation, orchestration architecture, product ownership, and knowledge-graph engineering. All build or operate productive agentic systems at SAP, giving between them a view that spans the description layer (ORD), the orchestration runtime (Joule), and the semantic grounding layer (Business Knowledge Graph). Table 8 summarises the panel. Each interview followed the same standardised guide, lasted roughly 45–60 minutes, and was conducted between 22 June and 24 July 2026.

A.2.1 Interview — Technical Product Manager

Role: Technical Product Manager in the AI Unit at SAP, responsible for Joule. *Date:* 22 June 2026.

Q1: What is your position? What are you currently building, and what is it meant to do?

I am a Product Manager in the AI Unit at SAP. I have been working with AI agents since March 2023, starting right after the release of ChatGPT and early frameworks like AutoGPT. Currently, I am working on Joule, which serves as a generalist

Table 8: Expert interview participants. All roles are at SAP; identifiers I1–I5 are used throughout the coding matrix (Appendix A.3) and the main text.

ID	Role and area	Date
I1	Technical Product Manager, AI Unit (Joule)	22 Jun 2026
I2	Principal Architect, Cross Product Architecture (ORD specification lead)	30 Jun 2026
I3	Area Architect, Joule Orchestration	01 Jul 2026
I4	Area Product Owner, Joule Orchestration	02 Jul 2026
I5	Lead Knowledge Graph Engineer, Business Knowledge Graph	24 Jul 2026

agent system. Joule is designed to orchestrate purpose-built expert agents and directly handle user requests across the entire SAP ecosystem, functioning as a broad-scale, overarching agent.

Q2: What do you see as the main challenges when building or adopting agentic systems in an enterprise context?

There are several major challenges. Organisationally, the innovation cycles are extremely fast, making it difficult to keep up and integrate new developments. Technically, the sheer size and breadth of SAP’s product portfolio is a massive hurdle. When trying to automate business processes across different products, you immediately run into data quality issues, as well as practical and conceptual problems.

For example, interpreting a user’s intent when they ask “Show me my open items” or “Show me my open sales orders” is incredibly difficult. Objects like “Sales Orders” exist in multiple different systems with overlapping definitions. Bringing the right systems together and accurately interpreting intent across such a diverse landscape is highly complex. Additionally, much of the necessary process knowledge currently exists only as undocumented tribal knowledge among employees. Finally, ensuring basic reliability, reproducibility, and safeguarding across these systems remains a constant challenge.

Q3: When you build an agentic solution, how do you decide what to build and what not to build?

We employ a hybrid approach: central teams provide the technology platform, while the Lines of Business build domain-specific expert agents. Initially, this was highly decentralised, which led to collisions, for instance, multiple expert agents claiming to handle “sales orders,” leading to ambiguity. We are now shifting towards a more centralised orchestration model to mitigate this.

Regarding use case selection, identification is largely left to the LoBs and is heavily based on perceived business value. However, the quality of these proposed use cases is quite mixed. As we learn to build agents, many proposed use cases are actually strictly deterministic processes. In those instances, it is often more appropriate to use standard automation rather than an agentic system. Currently, there is little systematic methodology to derive the implementation approach directly from the nature of the process itself.

Q4: How is process knowledge connected to the agent and the actual IT resources that execute the steps?

At the moment, process knowledge and the specific IT systems to be connected are mostly hardcoded by developers directly into the agent’s code. However, we are finding that this approach is insufficient, particularly for governance and authorisation. Consequently, developers are increasingly required to explicitly document and maintain separate lists of the specific systems and API endpoints their agents use. This allows our authorisation concepts to properly validate and grant access, ensuring that an agent acting on behalf of a user does not exploit all of the user’s privileges, but only the explicitly approved ones.

Q5: How are resources described today?

Resources are described using SAP’s Unified Meta Data Service via the Open Resource Discovery protocol. Systems and agents both have ORD identifiers. Currently, the wiring between an agent and a system is largely handled via manually created Agent Policies.

Q6: When an agent needs a resource at runtime, how does it find it?

The first step towards automated discovery is having Joule dynamically discover and orchestrate available systems and expert agents that are registered in ORD. We are exploring different retrieval approaches: one relies on self-describing text with minimal metadata, while another constructs a Knowledge Graph from ORD and other UMS metadata to facilitate the retrieval process and return the correct ORD IDs to the orchestrator.

Q7: How much structure does a resource description need to be reliably usable by an agent?

Standardised, central descriptions only go so far because customer landscapes rarely match textbook process models. Every customer has heavy customisations and unique local variants. Therefore, relying entirely on static, structural descriptions is

insufficient. The system needs enough structure to identify capabilities, but must rely heavily on dynamic adaptation to handle real-world deviations.

Q8: Do you see deterministic systems, agents and human tasks coexisting in your landscape?

From the orchestrator's perspective, there is no fundamental conceptual difference between a deterministic system (like an RPA or a standard API) and an agent. The orchestrator treats these target systems largely as black boxes. It focuses on what the system can achieve (its capabilities and outputs) rather than how it is implemented internally. The primary challenge in orchestration is not the type of system being called, but rather handling situations where multiple paths can achieve the same goal, and resolving the constraints to find the optimal route.

Q9: Would machine-readable self-descriptions be useful for an agentic orchestrator?

A highly scalable orchestrator cannot rely entirely on manual engineering, it is too complex and does not scale for thousands of customised enterprise scenarios. The architecture must evolve from using generic self-descriptions to becoming a self-optimising system.

Q10: If you had to build an orchestrator that can reliably disambiguate between similar resources, how would you approach it?

Rather than manually defining every step an agent should take, we should define the Definition of Good (target criteria and evaluations). By combining machine-readable metadata (like a Knowledge Graph) with feedback loops, the system can learn from its usage and adapt to local customer realities automatically. This self-optimising approach, fed by initial resource descriptions and continuous feedback, is the only viable way to achieve the required quality and coverage across a massive, customised ecosystem.

Q11: Further comments or questions?

No further comments. The topics discussed cover the core challenges and our current architectural direction.

A.2.2 Interview — Principal Architect

Role: Principal Architect in the Cross Product Architecture Team at SAP, responsible for the Open Resource Discovery (ORD) specification. *Date:* 30 June 2026.

Q1: What is your position? What are you currently building, and what is it meant to do?

I am a Principal Architect in the Cross Product Architecture Team, an organisation that drives architecture alignment across SAP. My focus areas are APIs and events, and specifically specifications and guidelines. The main artefact is the Open Resource Discovery specification, but we also own several other specifications. In general, we take industry standards such as OpenAPI and extend them so that SAP-internal use cases can be covered. We try not to create proprietary standards, but sometimes it becomes necessary because nothing exists yet, ORD itself is an example of that, and it is now around seven to eight years old.

I am also responsible for bridging topics: how to connect API and event architecture to the data product world, and how to connect it to the AI themes.

Q2: What do you see as the main challenges when building or adopting agentic systems in an enterprise context?

The first challenge is having a solid infrastructure foundation, for example, an Agent Gateway that enables centralisation. Simply installing MCP servers locally on every employee's machine will not scale in an enterprise context. You cannot govern that. Investment in good central infrastructure is therefore necessary.

Beyond infrastructure, governance becomes important: who has which permissions, do agents have different rights from the users who trigger them, how can actions be audit-logged, and how can company-wide policies be enforced? From a development perspective, companies also need to maintain an overview of what has been built, what quality standards it meets, and how those standards evolve over time. The entire lifecycle (from execution to continuous improvement) needs to be mastered, roughly the way organisations had to learn DevOps. We now need to do the same for AI.

Q3: When you build an agentic solution, how do you decide what to build and what not to build?

This belongs to the governance process around how we develop. One initial step is to start in planning: there is a planning tool where use cases for agents are captured, described, and reviewed with experts. For a company like SAP it is also important that not only technical and commercial criteria are considered, but also ethical criteria, before significant development begins. Sometimes the flow is reversed: POCs are built first and then moved into planning. If nothing comes of them, they are discarded. But if the goal is productisation, the use case must pass through a review process. This can be quite democratic,

in principle anyone can bring ideas. SAP has kept this process open so that a large number of ideas can emerge quickly, and filtering happens after the broad brainstorming phase.

Q4: How is process knowledge connected to the agent and the actual IT resources that execute the steps?

This belongs on the list of genuinely difficult enterprise governance topics. Getting agents to take company-specific processes into account is related to the audit-logging challenge. At a certain point, a degree of determinism must be introduced in order to maintain compliance.

From SAP's side, we understand standard processes well, and through tools like Process Mining via Signavio we can understand how processes typically run and benchmark which ones operate more efficiently. But ultimately, processes are something the customer defines and adapts. This means there must always be a mechanism for the customer to co-define, restrict, and control what happens. This is one of the hardest topics, and to my knowledge there are no good solutions yet, the whole industry is grappling with it.

To a degree it can be addressed at the API level: the existing backends and transactional logic that SAP already has in its systems can be handed to agents. If agents respect those boundaries, we can carry all of that into the agentic world. But where deterministic compliance is required, the enforcement must happen through APIs, access control, or workflows.

Q5: How are resources described today?

For tools, the current standard is MCP and the way tools are described within MCP. Thinking more broadly, tools are essentially the same as API operations, they can be described as an operation in OpenAPI for REST, or as an Entity Set with access information in OData, depending on the protocol. The key is that all of this is unified centrally through ORD, so that a central aggregator can provide a complete overview of all resources relevant to agents: APIs, native tools via MCP, skills, and other agents. There may also need to be an additional category for contextual information, such as an LLM wiki.

Q6: When an agent needs a resource at runtime, how does it find it?

Discovery currently works in two ways. First, through self-description: an agent or application can expose an ORD endpoint at runtime and describe the resources it provides. Second, for purely static information (such as skills) it is also possible to represent everything via GitHub repositories and handle the ORD publishing behind the scenes. Ideally, applications do not need to implement this themselves. It becomes part of frameworks. ORD has been used productively for discovery within SAP for several years and is now being rolled out much more broadly and with higher priority for the agentic world.

Q7: How much structure does a resource description need to be reliably usable by an agent?

The required structure is protocol-specific (OpenAPI operations for REST, MCP tool descriptions, or Entity Sets in OData) but the unifying factor is that everything ends up in ORD as the shared description layer. Beyond the protocol-level specification, the more important question is company-wide agreement on which metadata standards to use and how they are consistently published, so that a central aggregator can consume them without special-casing.

Q8: Do you see deterministic systems, agents and human tasks coexisting in your landscape?

Workflows are in practice often small microservices with API interfaces for triggering them and for retrieving their output or status. A workflow engine can therefore be described as a system with APIs. Workflow engines such as n8n also expose this via MCP, which brings them back into the same model we already have for other resources.

Q9: Would machine-readable self-descriptions be useful for an agentic orchestrator?

The ORD approach already in use at SAP implicitly confirms the value of machine-readable self-descriptions. Resources describe themselves via ORD endpoints, agents, applications, and MCP servers all expose their capabilities through this standardised description layer, and a central aggregator collects these descriptions and makes them available for discovery. SAP is actively expanding this rollout for the agentic context.

Q10: If you had to build an orchestrator that can reliably disambiguate between similar resources, how would you approach it?

For a reliable orchestrator, the prerequisite is a discovery layer and a company-wide agreement on which metadata standards to use (for example, how OpenAPI is used for REST APIs and how MCP is used within the company) and that all this information can be published consistently. If different applications and teams handle this differently, building a unified orchestration layer becomes very difficult.

As for the aggregator itself, in a very large company one likely reaches a point where the problem must be approached like a data warehouse or ETL challenge, so that the result is a performant layer that consolidates everything and provides a good API for consumers. The critical factor is less about which specific standards are chosen and more about having the

mechanisms in place at all. Once everything is collected centrally, the company also gains the ability to apply governance to what has been built, achieve observability, and continuously improve.

Q11: Further comments or questions?

No further comments.

A.2.3 Interview — Area Architect

Role: Area Architect within Joule, responsible for the Orchestration area. *Date:* 1 July 2026.

Q1: What is your position? What are you currently building, and what is it meant to do?

I am an Area Architect within Joule, responsible for the Orchestration area. Simplified, our task is the following: the user says something, and the system has to answer the question of what to do with that input. A simple request such as “show me tomorrow’s weather forecast” should be relatively straightforward: we have to find an API that returns the weather. But a single user statement can be arbitrarily complex, because it may decompose into multiple tools, tasks, and information sources: different tools, agents, information extracted from documents, a web search, and so on. As an architect, my role is to make sure that all of this comes together coherently, while at the same time supporting the various features that are also expected of the platform.

Q2: What do you see as the main challenges when building or adopting agentic systems in an enterprise context?

The biggest challenge is that we work with content from a wide range of organisations that we do not really control ourselves, or only to a limited extent. There are processes in place to review and validate this content, but realistically, when the content is authored by hundreds of different people, it looks very heterogeneous and does not always match how we envisioned it. As a platform, we can only ship a product that is as good as the content running on top of it, and that is a substantial problem in itself.

From an architecture perspective, the environment is exciting because technologies evolve so quickly, often within six or twelve months, that some things become obsolete, while others that are not yet technically reliable are already visible on the horizon. You cannot sell an enterprise customer on something that does not work today, but you also know that in six months it likely will. Keeping this balance is challenging.

Q3: When you build an agentic solution, how do you decide what to build and what not to build?

The realistic answer is: when the pressure to change is high enough, wherever that pressure comes from, then things get changed. And it is often too late. It is also difficult to throw away existing content, because a lot of people have invested significant time in it and have implemented use cases that they rely on continuing to work in exactly the same way. From an ideal perspective, that content may no longer make sense in a new approach, but the sunk-cost dynamics slow down the transition.

Formal processes for these decisions do not really exist at the moment, although this is starting to change as the Product Management organisation grows and contributes more perspective on what we actually want to support in the future, not just visionary directions, but concrete commitments. In the end, the user does not care whether an MCP server or a direct API sits under the hood. The decision should be driven by what actually creates value for the end user, balanced against what will still be technically relevant a year from now.

Q4: How is process knowledge connected to the agent and the actual IT resources that execute the steps?

Not directly addressed. Related aspects are covered under Q9 and Q10, where the interviewee argues that process knowledge should be part of a central graph rather than hardcoded into skills or agent code.

Q5: How are resources described today?

The main content-delivery concept in the current tool stack is the notion of Joule Functions, which split into so-called Scenarios and Dialog Functions. Scenarios carry the semantic metadata that describes what a given Dialog Function can do. To stay with the weather example, the scenario contains a description (“this is the weather scenario”), a more detailed description (“I can show you the weather for the next week”), and a list of slots such as location or day. The Dialog Function itself only handles the execution and calls the backend system. For orchestration, the scenarios are what matters. Today, we describe both agents and various tools through this scenario mechanism. It is the main way executable content enters the system, and executable content in practice is almost always wired through an API.

In addition to that, we have documents. Through Document Grounding, arbitrary text can be indexed, customer help documents that are exposed via SAP Help, policy documents, and effectively anything that internal groups such as SAP

IT consider knowledge-relevant. These are uploaded into Joule, indexed in chunks in Document Grounding, and retrieved through embedding search. Beyond that, there are custom tools available to the Tool Agent, for example a web search, but there is no formal delivery model for those yet, they are integrated ad hoc.

Scenarios are almost entirely authored by the Lines of Business (roughly 99.9%), with only a handful of defaults provided centrally. Customers can also register their own scenarios. If a customer wants to expose an internal API, for example a canteen menu endpoint, through Joule, they can write their own scenarios to bind that API. The same applies to non-SAP systems that a customer wants to integrate.

Q6: When an agent needs a resource at runtime, how does it find it?

The retrieval process distinguishes a pre-selection step from the actual orchestration decision. Pre-selection for both documents and scenarios runs on semantic similarity via embeddings. For documents, this happens at chunk level. For scenarios, similarity is computed between a rewritten user query and the scenario's description together with representative example sentences. If the user says "show me the weather", we may rewrite this into "show the weather for the user's current location" and match embeddings against scenario descriptions and document chunks. Pre-selection returns roughly ten scenarios plus a set of chunks, which then enters the core orchestration step. There, one or more LLM calls decide, based on that pre-selected information, which scenario to actually invoke and whether the retrieved document chunks are relevant at all. Once the scenario is selected, the underlying API is called.

There are ambitions to evolve this. What I described corresponds to the internal codename V-4. A follow-on line, currently referred to as V-10, moves toward a more classical agentic behaviour with a ReAct-style loop and a computer-using harness. The idea is to use progressive disclosure: the LLM is asked multiple times, each time seeing a different level of detail, and it can invoke additional tools, for instance a document-based question that returns further documents through embedding or other indexing methods. This direction is being pushed toward general availability. Part of the motivation is that user expectations have shifted. Developers have grown used to waiting longer for tools like Claude Code, and enterprise users of similar tools no longer expect a two-second answer for complex questions. That makes ReAct-style loops more realistic in practice than they were before.

Q7: How much structure does a resource description need to be reliably usable by an agent?

Implicitly addressed under Q5, Q6, and Q10. In the current V-4 stack, the minimum structure per scenario is a short description, a detailed description, a set of representative example sentences, and a slot list. This is what pre-selection matches against. Q10 argues that this baseline is insufficient for reliable disambiguation at scale and that semantically connected information over a graph is required in addition, so that relationships between resources (value helps, dependencies) become explicit rather than left to embedding similarity.

Q8: Do you see deterministic systems, agents and human tasks coexisting in your landscape?

Not directly addressed.

Q9: Would machine-readable self-descriptions be useful for an agentic orchestrator?

Yes, definitely. This aligns with the Knowledge Graph ambitions within SAP more broadly. In my view, the big problem with these systems is discovery: how do I find out which options are available to me at all, without having to pump everything I have into the LLM context, which is expensive, slow, and simply not enough. Doing this discovery efficiently and well is where the different approaches diverge, but in most cases they end up depending on having good, semantically connected information about the systems you are working with. ORD is one approach that carries partial semantic information and dependencies between resources, and at least parts of the discovery are expected to run through ORD, or through the Field Customer Landscape more broadly, so that discovery gets unified. Joule's orchestration is not the only stakeholder with this requirement: many other systems such as Signavio also need to know which APIs exist in a customer's landscape and what they actually look like. Extensibility is a huge topic for SAP customers, and it is what differentiates SAP from any arbitrary ERP startup: Nestlé does not simply run one SAP system out of the box.

Q10: If you had to build an orchestrator that can reliably disambiguate between similar resources, how would you approach it?

Ignoring cost constraints for a moment, the ideal system would have a fast-answer mode: an LLM path with minimal ceremony, no large sandbox, and short response time. But when a request looks business-relevant, several discovery processes should start in parallel to feed the actual reasoning harness, most likely a computer-using harness that can also write code and act on tools. Part of that discovery must return the available APIs, ideally via a knowledge graph, because this is not just a semantic question but also a relationship question: which API provides a value help for another one, which additional API can I offer alongside a given one, this is expressed most naturally over a graph.

Process information must also come out of such a graph, because as a user I want to describe the result I want, not the process that leads to it. The orchestrator should know that process itself. When I create a purchase order, Joule should know

what is needed to create a purchase order in our business system, including that, for example, an email has to be sent to my L+2 manager because that manager must approve every purchase order. I should not have to tell it to send the email as an additional step. That is the optimum. We are still fairly far away from it in many cases, but topics such as EKX and others are moving in that direction.

Alongside APIs and processes, information such as policies and company knowledge should be retrieved in parallel and handed to the harness as the starting point: here is what the user wanted, here is the base information, here are also skills that describe parts of these processes in more detail, please return the following result. Skills as they exist today are a step in that direction. They capture semantic knowledge of how things fit together. But strictly, that kind of knowledge should not be delivered through skills. It should sit centralised in a graph so the individual pieces are better connected. We are not there yet either. The typical architect's mindset applies: you are always fighting against constraints, and it is hard to describe the truly optimal world because you rarely have the chance to build it. That said, this direction would create the biggest value for users, even though under current technology it becomes very expensive: a single weather query at thirty cents is not something anyone wants to pay for.

Q11: Further comments or questions?

No further comments.

A.2.4 Interview — Area Product Owner

Role: Area Product Owner for Joule Orchestration at SAP. *Date:* 2 July 2026.

Q1: What is your position? What are you currently building, and what is it meant to do?

I have been Area Product Owner for Joule Orchestration for about two and a half months. The role covers portfolio work, driving innovation into the team, translating solution-oriented requirements aligned with Product Management into integration patterns and backlog items. I share the area with the Area Architect: he takes the architecture part, I take the Product Owner side for natural-language orchestration.

Q2: What do you see as the main challenges when building or adopting agentic systems in an enterprise context?

The biggest challenge in orchestration, and around it, is evaluation. With the rapid advancements in the AI space, many possible solutions surface, and patterns eventually stabilise in a certain direction, but the challenge is measuring how well they perform. In an SAP product like Joule, orchestration adds further complexity because it faces real-world requirements that go beyond laboratory conditions. A concrete example is multi-system: a customer may have the same system twice, which effectively adds an orchestration problem on top of the actual orchestration problem, so the complexity keeps multiplying.

Having a benchmark for each of these situations is what is missing today. There are many solutions, but identifying the technically best and best-evaluated one, translating it into a product roadmap, and handing the implementation to the teams is currently the biggest challenge.

Regarding evals as they exist today: there is no streamlined approach, they are quite scattered. There are Agent Evals, Knowledge Graph Evals in Joule, and prompt-level tests. On a test pyramid, prompt tests sit at the unit-test level: each LLM call in a product like Joule has a task-descriptive system prompt, and prompt tests define an expected output per LLM call and per orchestrator task. On that level the teams are reasonably well set up. It becomes harder in a distributed system where several specialised prompts talk to each other before converging on a final result. Communication overhead and complexity between the intermediate steps and the final result grow exponentially there. The most prominent internal tool for end-to-end testing today is Golden Dataset Studio, currently with a strong focus on the Knowledge Graph, working with boundary conditions of the form “as a user I input X and expect Y end-to-end”. Test cases are still curated manually, often by LoB-specific consultants, so statistically the data volume is small.

Q3: When you build an agentic solution, how do you decide what to build and what not to build?

For Skills or MCP the problem is mostly an integration problem rather than an orchestration problem, because most orchestration problems are already solved. It is the surrounding aspects that matter: integration, ownership, sustainability. MCP was top of mind maybe half a year ago and is already partially eclipsed by the ongoing discussion around alternatives like Agent Skills, even though the two can be complementary. From a product management perspective, the biggest challenge is keeping up with the market speed while maintaining product fit. This directly ties into a benefit-versus-effort question: can we afford to spend six months rebuilding Joule Orchestration for a new technology shift, knowing that in another six months something new may appear? Balancing this is essential, and the current process is largely: build a PoC, review it internally, run the unit-test-level prompt tests where applicable, hand it to LoBs for end-to-end feedback, and then decide together whether it warrants strategic productisation.

Q4: How is process knowledge connected to the agent and the actual IT resources that execute the steps?

There is a stark difference between standard processes that ship with SAP standard software and highly customised processes on the customer side. For standard processes, the first challenge is: where can it be found and how well is it consumable for an AI? An early step here was Conversational Search, which enabled a tool on SAP Help documentation and pulled relevant chunks via embedding retrieval into the LLM context to answer questions. This works for standard processes such as leave approval in SuccessFactors or a purchase-order creation, which are documented well enough for customers. It works less well when the content is not rich, when tables or figures are involved, or when formatting breaks the translation. Consumability for AI is then poor.

The next step was to solve this through a Knowledge Graph, though the current graph focuses less on process knowledge and more on the structure of backends and OData APIs. This will likely change and increasingly include process knowledge. The current Canonical Knowledge Graph, however, still only reflects SAP-standard knowledge. The actual value for a customer sits in their modified, customer-specific processes, and this requires a customer-specific or tenant-specific KG. That is technically challenging, and to my knowledge there is not yet a strong communication between the KG teams and the process-modelling side (Signavio) about how tenant-specific process knowledge would be accessed. EKX has strong Signavio integration, but it is Canonical, that is, SAP-standard. There is no scenario yet along the lines of “my Joule runs on Mercedes-Benz processes at Mercedes-Benz.” Around 90% of what we work on in AI is fundamentally integration, not the AI itself. The technical solutions largely exist, evaluation is the missing piece, and before evaluation, technical integration is the missing piece to enable this end-to-end.

Q5: How are resources described today?

In the classical tool stack that is not yet migrated to Agent Skills, resources are exposed through Tool Scenarios, which act as a semantic interface for a function. A scenario carries a name, a description, and parameters. This is essentially the JSON-based tool-calling standard used from the early days, treated here as a semantic interface. Currently, it is up to individual LoBs or content creators to decide what semantic information they place in a scenario. In practice, some LoBs treat this well. Others overload it, effectively scripting the interaction as “if the user says X, do Y, then say Z”, which is brittle and overfits to a specific prompt and model version. Scenarios in this stack are the mechanism for transactional operations such as posting a purchase order or creating a sales order. Each LoB uses the same framework to expose their backend functionality as a natural-language interface. On top of that, Conversational Search is modelled as a tool as well: it exposes a preview of chunks and offers the orchestrator the option to invoke this tool for informational use cases such as “what is the maximum lead distance for supplier XYZ”. Together, these two mechanisms provide the semantic surface on which the orchestrator makes its decision.

Q6: When an agent needs a resource at runtime, how does it find it?

For discovery, the orchestrator does not scan all tools. A Preselection step reduces the search space first. Preselection is one of the earliest mechanisms in Joule and was introduced when the context window was still around 16K tokens and everything had to be aggressively optimised. Preselection performs vector embedding matching between a context-rewritten user query, made independent of the conversation history, and each scenario description. It cuts down to the top N scenarios, and only these enter the orchestrator’s context for the final selection call.

Q7: How much structure does a resource description need to be reliably usable by an agent?

The observation across LoBs is that the amount of semantic content in a scenario description varies widely. Some LoBs describe their scenarios cleanly at the semantic level. Others write near-scripts inside the description, which overfits to a specific prompt and model. In either case, the orchestrator has only what the scenario description exposes, so quality bounds selection quality directly.

Q8: Do you see deterministic systems, agents and human tasks coexisting in your landscape?

Yes, they coexist. A Tool Scenario can encapsulate a state-machine-driven process, so once it is invoked the deterministic flow runs regardless. The problem is that the flow becomes semantically inflexible. A concrete example: SuccessFactors exposes a scenario “View Direct Reports”. If the user asks “How many direct reports does Dominik have?”, the orchestrator correctly picks the scenario. But the LoB implementation renders a list of direct reports rather than answering “you have twelve.” The scenario controls the conversation flow from the moment it is invoked but cannot do so context-aware, because it does not see the surrounding conversation. This is a legacy from earlier chatbot patterns where the tool function owned and drove the conversation flow itself. An LLM-based design that reasons over the state machine with full context would handle these cases better.

Q9: Would machine-readable self-descriptions be useful for an agentic orchestrator?

The idea is strong, provided it is adopted broadly and recognised as a standard across the ecosystem. If that happens, ORD

becomes consumable by essentially any AI client, which is a major advantage. The current issue is adaptation: as long as adoption is partial, the value is limited. Once the gap is closed, it becomes clearly attractive for Joule, because a client would get out-of-the-box access to semantic knowledge for many systems.

The open question is how tightly to bind the orchestrator to any single description standard, because the landscape has moved through several standards in quick succession (function-calling schemas, MCP, Agent Skills, agent CLIs). Architecturally, the orchestrator should sit at an abstraction level that can absorb such changes. That said, if a strong push commits to a standard such as ORD and it can be shown that Joule benefits from consuming it natively, it becomes a clear PoC candidate. The evaluation question comes back once more: if benefits can be quantified end-to-end, adoption follows.

Q10: If you had to build an orchestrator that can reliably disambiguate between similar resources, how would you approach it?

The direction the orchestrator moves in is a proper agent harness rather than the earlier chatbot-style pattern, including a computer-using variant of the harness (as pursued in the V-10 line). New possibilities open up in that direction: hybrid discovery that combines a filesystem search with additional retrieval steps, and using progressive disclosure to realise preselection through the agent loop itself rather than as a separate pre-orchestration stage. The trade-off is always cost versus benefit: the current preselection is optimised for speed because it runs on every request. Its role was to accelerate the easy, semantically obvious cases so the orchestrator does not have to run a full discovery loop every time. There is clearly appetite to change how this works, especially as Agent Skills arrive as a content format. The concrete algorithmic shape is not fixed, but the architectural direction is clear.

The broader challenge is doing product engineering in an environment where the technology shifts every few weeks. It requires a mindset shift, robust processes, and an architecture that can absorb changes rather than being rebuilt from scratch each time. Only then can the team iterate quickly. Today, before one release is out, the next paradigm is already surfacing, which makes sustainable product engineering very hard. Where in the Gartner Hype Cycle the industry currently sits is difficult to say. The peak seems unusually stretched, and the model itself may no longer apply cleanly.

Q11: Further comments or questions?

No further comments.

A.2.5 Interview — Knowledge Graph Engineer

Role: Lead Knowledge Graph Engineer at SAP, responsible for the Business Knowledge Graph (BKG) and its integration with Joule. The BKG provides semantically rich context for Joule to answer questions based on grounded SAP metadata.

Date: 24 July 2026.

Q1: What is your position? What are you currently building, and what is it meant to do?

Lead Knowledge Graph Engineer at the Business Knowledge Graph. Currently building the SAP Knowledge Graph and its integration with Joule. It is meant to provide semantically rich context for Joule to answer questions based on grounded SAP metadata.

Q2: What do you see as the main challenges when building or adopting agentic systems in an enterprise context?

The nondeterminism of agentic systems often works against what we are trying to achieve by standardising metadata in a single system. Agents can often see the data, but large context windows for complex queries reduce accuracy; running over multiple calls increases latency, which we are bound to commercially; and longer context increases cost. All of this affects commercial viability.

Q3: When you build an agentic solution, how do you decide what to build and what not to build?

Not directly building an agentic system in my current role.

Q4: How is process knowledge connected to the agent and the actual IT resources that execute the steps?

Process knowledge is often encoded in BPMN diagrams or other structured data formats. In the SAP Knowledge Graph, process knowledge is represented as nodes and edges connecting a process step to an application or API. These steps can be queried by an agent to better understand how to navigate best-practice processes and which APIs to execute.

Q5: How are resources described today?

Not directly owning an agent, but the project contributes to passing metadata to many agents. Resources are often described through skills generated directly from Knowledge Graph data to better guide the agent towards its target.

Q6: When an agent needs a resource at runtime, how does it find it?

At runtime an agent should have access to the list of tools within its own formation, scoped to the use cases it can solve. As tools grow in number and scope, and as responsibilities overlap, managing the tool discovery layer becomes critical to guide the agent to the best option while reducing cost from redundant calls. As more agents and tools are created, this creates a search space problem that must be handled by smart tool discovery design.

Q7: How much structure does a resource description need to be reliably usable by an agent?

Agents need data that is structured rather than just verbose text describing relations. If agents are running multi-step operations and reasoning over which paths to take, a good structure describing possible outcomes is relevant. In the SAP Knowledge Graph, agents (such as the BDC KG Reasoner) can query the graph and decide which path to continue in a step-wise fashion. This allows the agent to navigate complex landscapes with flexibility.

Q8: Do you see deterministic systems, agents and human tasks coexisting in your landscape?

Yes. The EKX platform already relies on a Knowledge Graph and agentic behaviour to serve and retrieve information, where humans in the loop can guide the agent with follow-up questions and the agents can retrieve data. This is a similar setup to the Knowledge Graph in Joule.

Q9: Would machine-readable self-descriptions be useful for an agentic orchestrator?

This scenario is already implemented and adopted at SAP through Open Resource Discovery (ORD) and the Unified Metadata Service (UMS). ORD transports and describes known business objects and APIs and relates them to each other; the UMS aggregates them and exposes them to consumers, the SAP Knowledge Graph being one of them. This approach helps standardise what agents see and how they interact with metadata across the company and its Lines of Business. It is scalable and allows standard sharing of agent resources, including skills. Current blockers are the pace of iteration on the UMS platform and accommodating varied data sources in a dynamic environment. The absence of central aggregators leads to contracting skills, heterogeneous formats, and eventually agentic confusion at scale. The overhead of adhering to quality rules may slow development, but it is a net win in the long term.

Q10: If you had to build an orchestrator that can reliably disambiguate between similar resources, how would you approach it?

The key requirement is a metadata schema attached to each resource that enables the agentic orchestrator to differentiate between resources in a meaningful way — enabling use-case search or process-based search rather than simple embedding approaches, which without extra semantic context become noisy when multiple resources have similar scope. The orchestrator should have multiple components built on top of a solid metadata registry that is consistent and queryable by the agent to determine its next step. Multiple search functions should be used: some based on processes, others covering a larger space using embedding approaches. For a long-lasting orchestrator, one should log and learn from the agent's mistakes; these logs can serve as future context (context graphs or similar) that guide the agent to avoid repeating errors and to better understand requirements over time. This not only makes the system more robust but also reduces cost and ensures that query patterns contribute to agent learning.

Q11: Further comments or questions?

No further comments.

A.3 Interview Coding Matrix

This appendix documents the qualitative content analysis applied to the expert interviews in the analysis chapter. The coding follows Mayring's combination of deductive and inductive categories [53]: the literature-derived requirements R1-L to R10-L (Appendix A.1) are used as deductive anchor categories, and additional themes raised by the interviewees are added inductively.

Each interview statement that touches a category is classified as **Confirm**, **Refine**, or **New**. *Confirm* if the statement supports the literature-derived requirement without modification, *Refine* if it sharpens, narrows or qualifies it, and *New* if it raises a concern that none of R1-L to R10-L covers. Findings are aggregated per theme rather than per interviewee. Across the five interviews, all ten literature-derived categories were confirmed at least once. Six of them were additionally refined with practitioner detail. Two themes appeared in single interviews and are noted as cross-cutting concerns rather than as new requirements. They did not constitute an additional R entry.

Illustrative statement-level coding (I1, I2). The aggregated matrix in Tab. 10 summarises the finding per category across all five interviews. To make the coding procedure transparent and reproducible, the following per-statement coding covers all five interviews (I1: Technical Product Manager, Joule. I2: Principal Architect, ORD protocol lead. I3: Area Architect, Joule Orchestration. I4: Area Product Owner, Joule Orchestration. I5: Knowledge Graph Engineer, SAP Business Knowledge

Graph). Each row records one condensed statement extracted from a passage of the transcript (Appendix A.2) and its verdict against the anchor category.

Table 9: Statement-level coding for all five interviews. I1: Technical Product Manager (Joule), I2: Principal Architect (ORD), I3: Area Architect (Joule Orchestration), I4: Area Product Owner (Joule Orchestration), I5: Knowledge Graph Engineer. Verdict: C = Confirm, R = Refine, N = New.

Cat.	Src	Paraphrased statement	V.
R1-L	I1	Democratic use-case intake with expert review and ethical criteria; POCs discarded if not productised — process spectrum handled by governance filter rather than by artefact type.	C
R1-L	I2	Many proposed agent use cases turn out to be strictly deterministic and are better solved by standard automation; no systematic methodology derives the implementation approach from process structure.	R
R1-L	I3	No formal process for deciding what to build; change is driven by external pressure. Product Management gaining influence to make future support commitments more explicit and less trend-driven.	R
R1-L	I4	Build decisions today are PoC-driven, evaluated internally via prompt tests and end-to-end feedback from LoBs, then jointly judged as strategically viable; there is no formalised requirement-derivation methodology.	R
R1-L	I5	Process knowledge is represented as nodes and edges in the Knowledge Graph connecting process steps to APIs; agents query this to navigate best-practice processes and identify which APIs to execute.	C
R2-L	I1	Skills treated as a first-class resource category alongside APIs, MCP tools, agents; potential additional category for contextual information (“LLM wiki”).	C
R2-L	I2	Process knowledge today mostly hardcoded into agent code; developers increasingly maintain separate lists of systems and endpoints — motivates modular externalisation.	R
R2-L	I3	Skills capture semantic knowledge of how things fit together and are a step in the right direction, but strictly should not be delivered as skills; central graph representation would be preferable so pieces are better connected.	R
R2-L	I4	Agent Skills as a content format will arrive as an addition to existing scenarios; how algorithms consume them alongside other formats is architecturally open, but the productive direction is a proper agent harness with progressive disclosure.	R
R2-L	I5	Resources are often described through skills generated directly from Knowledge Graph data to better guide the agent towards its target — a form of modular procedural knowledge derived from graph structure.	C
R3-L	I1	API and event architecture serve as the layer where existing backends and transactional logic can be handed to agents, aligning agentic execution with the implementation phase.	C
R3-L	I2	LoBs build domain-specific expert agents on top of a central platform; hybrid model targets the implementation phase but currently produces overlapping capabilities.	C
R3-L	I3	Joule Functions (Scenarios + Dialog Functions) operationalise the implementation phase: scenarios carry semantic metadata, dialog functions call the backend API.	C
R3-L	I4	Scenarios exposed by LoBs (create sales order, post purchase order, etc.) are how backend transactional operations enter the implementation phase; the orchestrator triggers them from natural-language input.	C
R4-L	I1	ORD used productively as central discovery layer for APIs, MCP tools, skills, agents; discovery via self-description or GitHub-based publishing behind frameworks.	C
R4-L	I1	At very large scale, discovery becomes a data-warehouse/ETL problem: aggregation needs a performant consolidated layer with a good consumer API.	R
R4-L	I2	Resources described via ORD (backed by SAP’s Unified Meta Data Service); dynamic discovery via ORD-registered systems and agents just starting with Joule.	C
R4-L	I2	Wiring today is largely manual through Agent Policies, which the ORD-based dynamic discovery is meant to replace.	R
R4-L	I3	Discovery is the core problem: too expensive and slow to pump all landscape data into the LLM context. Parts of discovery expected to run through ORD / Field Customer Landscape for unification across stakeholders (Joule, Signavio, others).	R
R4-L	I4	Preselection was introduced early because scanning all scenarios in one LLM call did not scale; today it cuts to top-N via embedding matching before the orchestrator runs the final selection call.	R
R4-L	I5	As tools grow in number and scope with overlapping responsibilities, managing the tool discovery layer becomes critical; the growing agent and tool count creates a search space problem requiring smart discovery design.	R
R5-L	I1	Explicit strategy: extend industry standards (OpenAPI, MCP) rather than create proprietary ones; ORD only where nothing exists yet.	C
R5-L	I2	ORD identifiers used uniformly for both systems and agents, enabling vendor-neutral referencing across the landscape.	C
R5-L	I3	Customer-authored scenarios can expose non-SAP APIs and internal endpoints through Joule; the model works vendor-agnostically.	C
R5-L	I4	The orchestrator should sit at an abstraction level above any single description standard, because the landscape moved through function-calling schemas, MCP, Agent Skills, and agent CLIs in short succession; binding it too tightly to one is risky.	R
R5-L	I5	ORD and UMS are already implemented at SAP to standardise what agents see across Lines of Business; the approach enables standard sharing of agent resources, including skills, in a scalable and vendor-agnostic way.	C
R6-L	I1	Tools $\equiv$ API operations (OpenAPI operation, OData entity set) — a purely syntactic view is insufficient; a central aggregator is needed to bring semantic information together.	C

Cat.	Src	Paraphrased statement (continued)	V.
R6-L	I2	Objects like “Sales Orders” exist across multiple systems with overlapping definitions; interpreting user intent across such a landscape is a core semantic problem.	R
R6-L	I3	Semantic pre-selection today runs on embedding similarity between rewritten queries and scenario descriptions plus example sentences; ORD carries partial semantic information and inter-resource dependencies.	R
R6-L	I4	The scenario name, description, and parameters constitute the semantic interface the orchestrator sees; when descriptions are overloaded with script-like directives, the layer no longer functions as a business-meaning layer but as prompt-specific instructions.	R
R6-L	I5	Agents need structured data rather than verbose text describing relations; use-case search or process-based search is needed because simple embedding approaches become noisy when multiple resources have similar scope without extra semantic context.	R
R7-L	I1	Ideally applications do not implement discovery themselves; ORD publishing is handled by frameworks and, for static content, via GitHub — minimising authoring effort.	C
R7-L	I2	Static, structural descriptions only go so far; heavy customer customisation forces the layer to be lightweight and to rely on dynamic adaptation.	R
R7-L	I3	Scenario metadata (description, detailed description, example sentences, slots) is a lightweight, LoB-authored layer; 99.9% comes from LoBs, defaults centrally provided.	C
R7-L	I4	Standard SAP process knowledge is documented (SAP Help), but consumability for AI is limited by tables, figures, and formatting that break well-known extraction. Customer-specific processes require a tenant-specific representation that current standards do not cover.	R
R7-L	I5	A metadata schema attached to each resource, forming a consistent and queryable registry, is the foundation for reliable orchestration; it must be harvestable and not require new authoring overhead.	C
R8-L	I1	Customer-specific process knowledge must connect to standard-process understanding (Signavio, Process Mining) and to API/backend enforcement — linkage is one of the hardest topics with no good industry solution yet.	R
R8-L	I2	Process knowledge and IT resources are currently hardcoded together in agent code; this is insufficient for governance and authorisation.	R
R8-L	I3	User wants to describe the desired result, not the process. Orchestrator should know that a purchase order also requires notifying the L+2 manager; process knowledge should sit in a central graph, not in the user prompt.	R
R8-L	I4	Tenant-specific process knowledge is the missing piece: EKX links Signavio well but only at Canonical SAP-standard level; there is no productive path today from a customer’s modelled processes to the runtime orchestrator (e.g., “my Joule runs on Mercedes-Benz processes”).	R
R8-L	I5	In the SAP KG, process knowledge is already represented as nodes and edges connecting process steps to applications and APIs, serving as a practical realisation of design-time process–IT linkage.	C
R9-L	I2	Multiple expert agents claiming to handle “sales orders” caused ambiguity in decentralised setup, motivating the shift to centralised orchestration.	R
R9-L	I2	Primary orchestration challenge is not the type of target system but resolving multiple paths that can achieve the same goal.	R
R9-L	I3	Discovery is fundamentally a relationship problem, not just a semantic one (which API is a value help for another, which co-selection makes sense); a knowledge graph is the natural representation.	R
R9-L	I3	Two-stage retrieval in production: embedding-based pre-selection returns roughly ten scenarios plus document chunks, followed by an LLM stage that picks and filters. Direction is toward ReAct loops with progressive disclosure (V-10).	R
R9-L	I4	Multi-system landscapes multiply the disambiguation problem: the same system present twice adds an orchestration layer on top of the actual orchestration problem, and current benchmarks do not systematically cover this.	R
R9-L	I4	Concrete failure of static tool descriptions in the current stack: SuccessFactors’ “View Direct Reports” is picked correctly but the LoB implementation renders a list, not the numeric answer the user asked for; the scenario cannot adapt to conversational context.	R
R9-L	I5	Multiple search functions should be combined: some based on process knowledge, others covering a larger space using embedding approaches; neither alone is sufficient to disambiguate dense action spaces reliably.	R
R10-L	I1	Company-wide agreement on metadata standards (how OpenAPI/MCP are used) is the precondition; without it, unified orchestration cannot be built.	C
R10-L	I2	Reliability, reproducibility and safeguarding are constant challenges; poor quality in proposed use cases and ambiguous descriptions cause routing errors.	C
R10-L	I3	Content authored by hundreds of contributors across LoBs is inherently heterogeneous; platform quality bounded by content quality, even with review processes in place.	C
R10-L	I4	LoBs overload scenario descriptions with prompt-specific scripts (“if user says X, do Y”), which overfits to a specific prompt and model version and breaks under changes; this is a systemic description smell rather than a per-tool issue.	R
R10-L	I5	Without extra semantic context, embedding-based retrieval becomes noisy when multiple resources have similar scope; description quality directly bounds disambiguation quality.	R

Inductively identified themes (no matching R-L)

N1	I1	Governance, permissions and audit logging: agents may hold different rights from users, actions must be audit-logged, and company-wide policies enforced.	N
N1	I2	Authorisation must be scoped so an agent cannot exploit all of a user’s privileges; enforces explicit resource lists.	N

Cat.	Src	Paraphrased statement (continued)	V.
N2	I1	Self-optimising orchestrator with “Definition of Good” and feedback loops as the only viable path to scale — architectural direction rather than a requirement on the description layer.	N
N2	I5	Logging and learning from agent mistakes via context graphs or similar mechanisms enables the agent to avoid repeating errors and understand requirements better over time — a practical realisation of the self-optimising direction.	N
N3	I3	Fast-answer versus business-relevant modes: a dual-mode orchestrator with a lightweight LLM path for trivial queries and a parallel, computer-using discovery pipeline for business-relevant requests.	N
N3	I3	User expectations on response time have shifted; enterprise users now tolerate longer waits (Claude Code, Perplexity), making ReAct-style loops with progressive disclosure practically viable.	N
N4	I4	End-to-end evaluation is the bottleneck: prompt tests cover unit level well, but distributed orchestration between specialised prompts scales exponentially in complexity; no shared benchmark exists for multi-system and tenant-specific cases.	N
N5	I4	Product engineering at hype-cycle speed: robust architectures must absorb standard shifts (function-calling, MCP, Skills, CLIs) without full rebuilds; balancing new-technology adoption against sustainable delivery is a first-class product concern.	N

Across all five interviews, several themes recur without a matching R-L. First, governance, permissions and audit-logging (I1, I2) is retained as cross-cutting concern N1 in Tab. 10 below. Second, self-optimising orchestration with feedback loops (I1, I5) is treated as a future-work direction rather than a description-layer requirement: I1 describes the “Definition of Good” feedback-loop architecture, and I5 independently describes logging agent mistakes as context graphs for future guidance. Third, I3 raises a dual-mode orchestrator (fast-answer vs. business-relevant with parallel discovery) and shifted user expectations on response time that make ReAct loops with progressive disclosure practically viable. Fourth, I4 adds two related concerns: end-to-end evaluation as the current bottleneck for distributed orchestration across specialised prompts, and product engineering at hype-cycle speed, which requires the orchestrator to sit at an abstraction level that can absorb rapid changes in description standards. These architectural directions inform the design space but do not constitute additional description-layer requirements.

Coder reliability. The coding was performed by a single coder. Following Mayring, an independent second coder would strengthen inter-rater reliability; this was not feasible within the scope of the thesis and is noted as a methodological limitation. To mitigate this, the coding rules (Confirm / Refine / New) were fixed *before* any interview was conducted and are anchored in the pre-defined literature-derived categories R1-L to R10-L (Appendix A.1), reducing coder degrees of freedom.

Table 10: Coding matrix mapping interview findings to literature-derived requirements (R1-L to R10-L). All ten categories were confirmed; refined entries carry additional practitioner detail. Sources listed as interview identifiers.

Category	Type	Aggregated finding (Sources)
R1-L	Confirm + Refine	Agent-based automation is feasible across the structured-to-knowledge-intensive process spectrum in principle, but the in-scope boundary in current SAP landscapes is drawn at recurring, semi-structured steps rather than at fully knowledge-intensive cases or rigid end-to-end flows. (I1, I2, I3, I5)
R2-L	Confirm + Refine	Modular, dynamically loadable units of procedural knowledge are a recognised pattern; practitioners describe today’s substitutes (runbooks, prompt fragments, SOPs) as the closest existing analogue. Strong caveat from an orchestration-architect perspective: skills capture the right kind of semantic knowledge, but ideally that knowledge should sit in a central graph rather than be shipped as file-based skill artefacts, so that inter-piece relationships remain explicit. Product-side view (I4) adds that Agent Skills will arrive as a content format alongside existing scenarios, so the algorithmic question is how the orchestrator consumes multiple formats through one abstraction. (I2, I3, I4, I5)
R3-L	Confirm	The implementation phase of the BPM lifecycle is the natural entry point for agentic execution, and practitioners view it as the layer where today’s manual integration effort would shrink most. (I1, I2, I3, I4)
R4-L	Confirm + Refine	Dynamic discovery through a registry-like mechanism is needed, and flat MCP-style catalogues are reported as the dominant pain point in current landscapes; discovery is consistently described as the single biggest problem, with multiple stakeholders (Joule, Signavio, others) requiring the same unified view of the customer landscape. (I2, I3, I4, I5)
R5-L	Confirm	Loose coupling through open, vendor-neutral interfaces is a precondition for adoption; bespoke per-vendor adapters are reported as the main barrier today, and customer-authored descriptions must be able to expose non-SAP APIs through the same layer. (I1, I2, I3, I4, I5)
R6-L	Confirm + Refine	Machine-interpretable semantics beyond syntax is needed; practitioners report that current descriptions oscillate between free-text docstrings, script-like prompt directives, and proprietary metadata, none of which is reliably usable by an LLM-based orchestrator. Description overload with prompt-specific instructions is a recurring failure mode. (I1, I2, I3, I4)

Category	Type	Aggregated finding (continued)
R7-L	Confirm + Refine	A lightweight description layer that is harvestable from existing artefacts (OpenAPI, BPMN, internal catalogues) is strongly preferred over any new authoring effort; the SBPM-style ambition is endorsed conceptually but only on condition that it does not impose an ontology stack. Consumability of existing artefacts for AI is limited where formatting (tables, figures) breaks extraction, and tenant-specific process knowledge is not covered by canonical representations. (I1, I3, I4, I5)
R8-L	Confirm + Refine	Design-time linkage between BPMN/CMMN artefacts and IT resources is reported as missing in practice; current linkages, where they exist at all, are maintained manually and break at the first landscape change. The user's intent is to describe a result, not a process; the orchestrator should carry the process knowledge (e.g., that a purchase order also requires L+2 manager approval) rather than expecting the user to specify it. Canonical (SAP-standard) linkages such as EKX-Signavio exist, but there is no productive path yet from a customer's tenant-specific process models to the runtime orchestrator. (I1, I3, I4)
R9-L	Confirm + Refine	Disambiguation in dense action spaces is a recurring concrete pain; examples include overlapping MCP servers for the same domain and multiple internal agents exposing similar capabilities, and multi-system landscapes (the same system present twice at a customer) that add an orchestration layer on top of the actual orchestration problem. Two-stage retrieval is already the productive pattern (embedding pre-selection followed by an LLM decision), but the underlying problem is fundamentally about relationships between resources (value helps, dependencies, co-selection), which motivates graph-based rather than purely embedding-based representations. (I2, I3, I4, I5)
R10-L	Confirm + Refine	Description smells (unclear, redundant, misleading descriptions) are recognised as a real problem; practitioners report cases where two functionally identical tools are selected differently depending on description quality alone. Content quality bounds platform quality even where review processes exist, because content is authored by hundreds of contributors across many organisational units. A recurring concrete smell is LoBs overloading scenario descriptions with prompt-specific scripts, overfitting the interaction to a single prompt and model version. (I2, I3, I4, I5)
<i>Inductively extracted themes (cross-cutting, not promoted to requirements)</i>		
N1	Cross-cutting	Governance, permissions, and audit logging: agents may hold different rights from users, actions must be audit-logged, and company-wide policies enforced. Treated as a cross-cutting design concern rather than a description-layer requirement. (I1, I2)
N2	Cross-cutting	Self-optimising orchestration with feedback loops: the orchestrator learns from usage to adapt to local customer realities. Treated as a future-work direction rather than a description-layer requirement. (I1)
N3	Cross-cutting	Dual-mode orchestrator with fast-answer versus business-relevant paths, and shifted user tolerance for longer response times (Claude Code, Perplexity) that legitimises ReAct loops with progressive disclosure. Informs cost–accuracy trade-offs in the evaluation. (I3)
N4	Cross-cutting	End-to-end evaluation as the current bottleneck for distributed orchestration across specialised prompts; no shared benchmark for multi-system and tenant-specific cases. (I4)
N5	Cross-cutting	Product engineering at hype-cycle speed: robust architecture must absorb rapid standard shifts (function-calling, MCP, Skills, CLIs) without full rebuilds. (I4)

A.4 Final Requirements Catalogue

This appendix presents the final requirements catalogue R1–R10 in its detailed form. A compact summary appears in the main paper. The catalogue is the consolidated output of the two-stage derivation: the literature-derived requirements R1-L to R10-L (Appendix A.1) were validated and, where applicable, refined through the expert interviews (Appendix A.3). Each entry below records the consolidated requirement, the rationale tying it back to the four enterprise problems P1–P4, and where the interviews added practitioner refinement.

Two inductively extracted themes did not produce additional R entries but are noted as cross-cutting concerns:

- **N1 (Audit trail and explainability):** routing decisions must be inspectable. Treated as a design concern realised through the decision trace of the runtime flow.
- **N2 (Role-based access on resources):** fine-grained access control on resources. Treated as an evaluation-scope concern and out of scope for the artefact.

Table 11: Final requirements catalogue R1–R10 with rationale and interview-derived refinement notes.

ID	Requirement & Rationale	Interview Refinement
DO1 (Process Execution)		
R1	Full process spectrum coverage. The artefact must address processes from structured, repeatable workflows to unmodelled, knowledge-intensive cases. Driven by P4 (coexistence) and the BPM process spectrum (A-BPMS, SBPM).	Refined: in-scope boundary in current SAP landscapes is drawn at recurring, semi-structured steps rather than at fully knowledge-intensive cases or rigid end-to-end flows (I1, I2, I3, I5).
R2	Modular procedural knowledge (skills). Procedural knowledge must be representable as modular, dynamically loadable units. Driven by P2 (action-space complexity) and agentic BPM skill artefacts.	Confirmed: today’s substitutes (runbooks, prompt fragments, SOPs) are the closest existing analogue (I2, I3, I5).
R3	Agentic execution in BPM implementation phase. The implementation phase of the BPM lifecycle is the natural entry point for agentic execution. Driven by P4 (coexistence) and agentic BPM literature.	Confirmed without modification (I1, I2, I4).
DO2 (Description Layer)		
R4	Registry-mediated dynamic discovery. Resources must be discoverable through a registry-like mechanism with internal, typed structure. Driven by P1 (agent sprawl), P2 (action-space complexity) and service-discovery literature (SOA, ORD).	Refined: flat MCP-style catalogues are the dominant pain point in current landscapes (I2, I3, I4, I5).
R5	Vendor-neutral interfaces. Open, vendor-neutral interfaces must enable interoperability with previously unknown parties. Driven by P1 (agent sprawl) and service description standards literature.	Confirmed: bespoke per-vendor adapters are the main barrier today (I1, I2, I4, I5).
DO3 (Semantic Enrichment)		
R6	Machine-interpretable semantics. Descriptions must carry the meaning of a resource in business terms, not only invocation syntax. Driven by P3 (semantic specification gap) and semantic service description literature.	Refined: current descriptions oscillate between free-text docstrings and proprietary metadata, neither reliably usable by an LLM-based orchestrator; without extra semantic context, embedding-based retrieval becomes noisy when multiple resources have similar scope (I1, I2, I3, I4, I5).
R7	Lightweight, harvestable description layer. The SBPM ambition is retained without the heavyweight ontology stack; the layer must be harvestable from existing artefacts. Driven by P3 (semantic specification gap) and SBPM literature.	Refined: any new authoring effort is rejected; SBPM-style ambition endorsed conceptually only if no ontology stack is imposed (I1, I3, I4, I5).
R8	Design-time process–IT linkage. BPMN/CMMN process artefacts must be linked to IT resources at design time. Driven by P3 (semantic specification gap) and A-BPMS/skills literature.	Refined: linkage is missing in practice; where it exists, it is manual and breaks at the first landscape change. The SAP KG already realises this linkage as nodes and edges connecting process steps to APIs (I1, I3, I4, I5).
DO4 (Orchestration Layer)		
R9	Disambiguation in dense action spaces. Selection must work in dense action spaces with overlapping descriptions. Driven by P2 (action-space complexity) and tool retrieval literature.	Refined: overlapping MCP servers and multiple internal agents exposing similar capabilities are concrete recurring pains (I2, I3, I4, I5).
R10	Robustness against description smells at the Semantic Layer. The Semantic Layer must be robust against unclear, redundant, or misleading descriptions. Driven by P1 (agent sprawl), P2 (action-space complexity) and description-quality findings in the tool retrieval literature.	Confirmed: two functionally identical tools are selected differently depending on description quality alone (I2, I4, I5).

B. Design Artefacts

This appendix details the artefacts that realise the reference architecture: the ORD-based description and semantic layers, a representative skill artefact, the full specification of the seven retrieval methods, the design-time and runtime data flows, and the mapping of all requirements onto the architectural elements that satisfy them.

B.1 Open Resource Discovery and the Apeiro Stack

This appendix gives the ORD background this paper builds on. A full treatment of the protocol, its governance, and its place in the Apeiro Reference Architecture is given in the companion benchmark paper; here it is condensed to what the orchestration architecture depends on.

Apeiro stack. ORD is one component of the Apeiro Reference Architecture contributed by SAP to the IPCEI-CIS [32]. Four components form a Data Fabric for enterprise landscapes: Open Resource Discovery (ORD) as the metadata publication protocol, the Unified Customer Landscape (UCL) as the ORD Aggregator that crawls all providers and consolidates them into a queryable landscape model, the Unified Metadata Service (UMS) as the central query endpoint on top of UCL, and a Knowledge Graph that layers AI-derived semantic relationships over the aggregated metadata. UMS and the Knowledge Graph are not yet publicly available; this work therefore operates directly on the harvestable ORD layer. ORD itself is a machine-readable protocol that lets systems self-describe their resources: each implements a read-only Service Provider Interface, and an aggregator re-exposes the crawled result as a connected landscape. Beyond APIs and events, ORD covers Entity Types, Data Products, Integration Dependencies, and Capabilities, grouped through Packages and Consumption Bundles and related through typed links.

Concrete entry format. Listing 1 shows a representative ORD document for the Dispute Resolution use case. The document root carries the protocol version, the policy levels, and the `describedSystemInstance` that anchors all descriptions to a specific tenant. It then contains four resource types that illustrate the key concepts: an `agent`, an `apiResource` for the A2A endpoint, an `integrationDependency` naming an upstream service, and a `package` grouping them.

Across all resource types, the `ordId` is the globally unique identifier under which other resources reference an entry, and entity-type links such as the cross-system ODM type `sap.odm:entityType:Dispute:v1` express shared semantic references that the retrieval methods exploit.

Namespaces and shared references. Every ORD identifier carries a namespace that expresses ownership of the definition, not its location. An *authority namespace* (e.g. `sap.odm`) lets a cross-system component govern one contract that multiple systems implement, so the same governed definition is published from several systems under a single identifier rather than duplicated per publisher. This is what keeps cross-resource references (`integrationDependencies`, `exposedApiResources`, entity-type links) resolvable as the landscape grows, and it is the property the shared `sap.odm` entity-type vocabulary relies on. The benchmark’s landscape is built directly on this pattern.

B.2 Representative Skill Artefact

Skill artefacts are generated from the design-time flow and stored as markdown files in `benchmark/test_cases/design_time/output/skills/` within the benchmark repository. The example in Listing 2 is based on the CMMN process `proc_023` (`proc_ProductionTransitionWorkforce_v1`), a production transition workflow that coordinates workforce readiness, equipment validation, and customer communication. Each file contains YAML front-matter with a machine-readable description and an ordered list of steps. Steps that were matched to ORD resources at design-time carry a confirmed `ord_confirmed` annotation; unmatched steps remain as plain text. At runtime, the Intent Resolver sees only the `name` and `description` fields, and the Planner then loads the full steps of the selected skill.

B.3 Retrieval Methods

Implementation of the seven retrieval methods (Baseline, Embedding, Progressive, Graph, Agentic Tools, Agentic Raw, Agentic Hybrid). All share the same interface: query ℓ , ORD landscape R (clean x_0 or enriched x_1), result size k , return a ranked list or $\emptyset$. In the pseudocode, `LLM(.) = claude-haiku-4-5`, `Embed(.) = text-embedding-3-large`, `Cosine(., .) = cosine similarity`.

B.3.1 Free-text Baseline

Free-text Baseline is implemented in `src/methods/method_s.py` and serves as the no-retrieval reference against which the structured methods (Embedding Cosine, Progressive Disclosure, Graph Walk, Agentic Tools, Agentic Raw, Agentic Hybrid) are compared. It bypasses every retrieval mechanism (embeddings, filtering, graph traversal, agentic loops)

Listing 1: Representative ORD document for the Dispute Resolution use case (adapted from [31, 32]). The document root anchors descriptions to a system instance; agent, API resource, integration dependency, and package entries illustrate cross-resource referencing.

```

1 {
2   "openResourceDiscovery": "1.15",
3   "policyLevels": ["sap:core:v1", "sap:ai-agent:v1"],
4   "describedSystemInstance": {
5     "baseUrl": "https://tenant1.example-sap-system.com"
6   },
7   "agents": [{
8     "ordId": "sap.foo:agent:disputeAgent:v1",
9     "title": "Dispute Resolution Agent",
10    "shortDescription":
11      "AI agent specialized in financial dispute case resolution",
12    "version": "1.0.3",
13    "partOfPackage": "sap.foo:package:ord-reference-app:v1",
14    "relatedEntityTypes": [
15      "sap.foo:entityType:DisputeCase:v1",
16      "sap.foo:entityType:DisputeResolution:v1",
17      "sap.odm:entityType:Dispute:v1" ],
18    "exposedApiResources": [
19      {"ordId": "sap.foo:apiResource:DisputeResolutionAgent:v1"}],
20    "integrationDependencies": [
21      "sap.foo:integrationDependency:DisputeCaseManagement:v1",
22    "partOfGroups": [
23      "sap.aicore:llmModel:sap.aicore:anthropic--claude-3.7-sonnet"],
24    "partOfGroups": ["sap.eab:process:sap.foo:DisputeResolutionProcess"],
25    "labels": {"foo.ai:interaction-mode":
26      ["conversational", "system-triggered"]},
27    "tags": ["finance", "dispute-resolution", "ai-agent"]
28  }],
29  "apiResources": [{
30    "ordId": "sap.foo:apiResource:DisputeResolutionAgent:v1",
31    "title": "Dispute Resolution Agent",
32    "apiProtocol": "a2a",
33    "partOfPackage": "sap.foo:package:ord-reference-app:v1",
34    "resourceDefinitions": [{
35      "type": "a2a-agent-card",
36      "mediaType": "application/json",
37      "url": "/definitions/DisputeResolutionAgentcard.json",
38      "accessStrategies": [{"type": "open"}]
39    }],
40    "partOfConsumptionBundles": [
41      {"ordId": "sap.joule:consumptionBundle:mTLS:v1"}],
42    "exposedEntityTypes": [
43      {"ordId": "sap.foo:entityType:DisputeCase:v1"},
44      {"ordId": "sap.odm:entityType:Dispute:v1"}]
45  }],
46  "integrationDependencies": [{
47    "ordId":
48      "sap.foo:integrationDependency:DisputeCaseManagement:v1",
49    "title": "Dispute Case Management Integration",
50    "mandatory": true,
51    "aspects": [{
52      "apiResources": [
53        {"ordId": "sap.bar:apiResource:DisputeResolutionAPI:v1"}]
54      }
55    ]
56  }],
57  "packages": [{
58    "ordId": "sap.foo:package:ord-reference-app:v1",
59    "title": "ORD Reference Application Package",
60    "version": "1.0.0",
61    "vendor": "sap:vendor:SAP:"
62  }]
63 }

```

and issues a single LLM call that receives the query together with a flat catalogue of resources. Each resource is rendered as a single line containing only its `ordId`, `title`, and `shortDescription`, mirroring what an MCP or A2A tool catalogue looks like in practice. The LLM returns the top- k `ordIds` it considers most likely relevant. Since Free-text Baseline has no explicit refusal channel and no scoring signal beyond the LLM’s ranking, an empty return is not possible by design. Free-text Baseline therefore isolates what the language model alone can extract from unstructured, free-text tool descriptions and provides the lower bound for methods that add structural information on top. Algorithm 1 gives the pseudocode. See App. C.1, Listing 4 for an example trace on a design-time activity.

B.3.2 Embedding Cosine

Embedding Cosine is implemented in `src/methods/method_a.py`. Its core logic consists of two functions. `_resource_text(r)` builds the embedding input for a resource by concatenating up to seven fields with a `|` separator: `title`, `shortDescription`, and `description` are always included because they form the stable free-text

Listing 2: Representative skill artefact for `proc_023` (CMMN, `proc_ProductionTransitionWorkforce_v1`). Steps with `ord_confirmed` carry pinned `ordIds` and bypass retrieval at runtime; unmatched steps remain as plain text and are resolved dynamically.

```

1  ---
2  name: proc_023
3  description: >
4      This skill orchestrates production transitions triggered by engineering
5      changes, coordinating workforce readiness assessments, equipment
6      validation, asset provisioning, and customer communications.
7  metadata:
8      process-id: proc_023
9      process-type: cmmn
10     landscape-version: v2-benchmark-273
11  ---
12
13  ## Steps
14
15  ### Step 1: Analyze Engineering Change Impact and Workforce Readiness
16  <!-- ord_confirmed: sap.s4:dataProduct:EngineeringChangeImpactAndWorkforceReadiness:v1 -->
17  **Capability:** analyze-change-impact
18
19  Evaluate how the engineering change affects production operations and
20  identify skill gaps in your current workforce.
21
22  ### Step 2: Validate Material and Equipment Specifications
23  **Capability:** validate-specifications
24
25  Retrieve and verify all material definitions and vendor specifications
26  impacted by the engineering change.
27
28  ### Step 3: Assess Equipment Service Readiness and Dependencies
29  <!-- ord_confirmed: my.mes:apiResource:EquipmentServiceAndChangeManagement:v1 -->
30  **Capability:** assess-equipment-readiness
31
32  Evaluate the current condition and service status of production equipment
33  while identifying cascading dependencies across the manufacturing line.

```

Algorithm 1 Free-text Baseline (`src/methods/method_s.py`)

Require: query label ℓ , resources R , top- k

```

1:  $C \leftarrow []$ 
2: for all  $r \in R$  do
3:    $C.append(r.ordId || r.title || r.shortDescription)$ 
4: end for
5:  $ids \leftarrow LLM(\ell, C, k)$  {single call, returns top- $k$  ordIds}
6: return  $ids$ 

```

core of every ORD entry. `partOfGroups` (process group IDs), `processNext` (successor ORD IDs), `capabilities` (verb-noun labels), and `useCases` (natural-language scenarios) are appended only when present, since they are semantic fields that exist only on enriched ORD. The pipe separator is chosen over whitespace concatenation to preserve field boundaries in the embedding, which prevents the model from treating adjacent field values as a single phrase. On clean ORD (x_0), only the first three fields are populated. On enriched ORD (x_1), all seven fields contribute, which broadens the semantic surface the embedding sees. `retrieve()` calls `llm.embed()` for the query label and for each resource’s text string, then computes cosine similarity manually via dot product and L2 norm.

The manual implementation avoids an external vector-library dependency and makes the scoring step transparent in the code. Embedding calls are transparently disk-cached by `(model, text)` hash in `src/core/llm.py`, so repeated calls for the same resource text return at zero latency and zero incremental token cost. To reflect production deployment, the reported per-case token cost only accounts for the query embedding, while the resource embeddings are treated as a one-time landscape-setup cost (logged separately as `landscape_setup_tokens`). Embedding Cosine always returns the top- k candidates ranked by descending cosine score. Refusal is not possible by design, because the method has no stopping criterion and always has a ranked list to return, even if all scores are low. Algorithm 2 gives the pseudocode. See App. C.1, Listing 4 for an example trace on a design-time activity.

B.3.3 Progressive Disclosure

Progressive Disclosure is implemented in `src/methods/method_b.py`. It filters the landscape through up to four sequential LLM calls, each receiving a dedicated system prompt that constrains its decision to one dimension of the ORD hierarchy. The ordering follows a structural-before-semantic principle: cheapest and most discriminative filters first, most expensive last.

Algorithm 2 Embedding Cosine (src/methods/method_a.py)

Require: label ℓ , landscape R , result size k

Ensure: ranked list of $\leq k$ candidates

```
1: function ResourceText( $r$ ):
2:   parts  $\leftarrow$  [ $r$ .title,  $r$ .shortDescription,  $r$ .description]
3:   if  $r$ .partOfGroups  $\neq \emptyset$  then
4:     append "partOfGroups: " + join( $g$ .groupId for  $g \in r$ .partOfGroups)
5:   end if
6:   if  $r$ .processNext  $\neq \emptyset$  then
7:     append "processNext: " + join( $r$ .processNext)
8:   end if
9:   if  $r$ .capabilities  $\neq \emptyset$  then
10:    append "capabilities: " + join( $r$ .capabilities)
11:   end if
12:   if  $r$ .useCases  $\neq \emptyset$  then
13:    append "useCases: " + join( $r$ .useCases)
14:   end if
15:   return join(non-empty parts, " | ")

16:  $\mathbf{q} \leftarrow \text{Embed}(\ell)$  {disk-cached by hash(model,  $\ell$ )}
17: scored  $\leftarrow []$ 
18: for all  $r \in R$  do
19:    $\mathbf{v}_r \leftarrow \text{Embed}(\text{ResourceText}(r))$  {cached; zero cost on repeat}
20:    $s_r \leftarrow \mathbf{q} \cdot \mathbf{v}_r / (\|\mathbf{q}\| \cdot \|\mathbf{v}_r\|)$  {manual dot-product / L2 norm}
21:   scored.append(( $s_r$ ,  $r$ .ordId))
22: end for
23: sort scored descending by  $s_r$ 
24: return [{ordId, score} for top- $k$  entries]
```

Algorithm 3 Progressive Disclosure (src/methods/method_b.py)

Require: label ℓ , landscape R , result size k

Ensure: ranked list of $\leq k$ candidates, or $\emptyset$ if refused

```
1:  $N \leftarrow \text{NsSummary}(R)$  { $\{\text{ns} \rightarrow \{\text{count}, \text{top\_ets}, \text{lobs}\}\}$ , sys:  $\_\text{NS\_SYS}$ }
2:  $\hat{N} \leftarrow \text{LLM}(\ell, N)[1:2]$  {up to  $\text{MAX\_NAMESPACES} = 2$ }
3: if  $\hat{N} = \emptyset$  then
4:   return  $\emptyset$ 
5: end if
6:  $R_1 \leftarrow \{r \in R \mid r.\text{namespace} \in \hat{N}\}$ 
7: modCounts  $\leftarrow$  {operational: |agents+APIs in  $R_1$ |, analytical: |dataProducts+events in  $R_1$ |}
8:  $\hat{M} \leftarrow \text{LLM}(\ell, \text{modCounts})$  {sys:  $\_\text{MOD\_SYS}$ ; fallback: all present mods}
9:  $R_2 \leftarrow \{r \in R_1 \mid r.\text{type} \in \text{MODALITIES}[\hat{M}]\}$ 
10: if  $\exists$  entity types in  $R_2$  then
11:   etCounts  $\leftarrow$  entity-type frequency in  $R_2$  {only reachable ETs shown}
12:    $\hat{E} \leftarrow \text{LLM}(\ell, \text{etCounts})$  {sys:  $\_\text{ET\_SYS}$ ;  $\emptyset$  = pass-through}
13:    $R_3 \leftarrow \{r \in R_2 \mid r.\text{entityTypes} \cap \hat{E} \neq \emptyset\}$  if  $\hat{E} \neq \emptyset$  else  $R_2$ 
14:   if  $R_3 = \emptyset$  then
15:      $R_3 \leftarrow R_2$  {never filter to zero}
16:   end if
17: else
18:    $R_3 \leftarrow R_2$  {Stage 3 skipped; total calls = 3}
19: end if
20:  $C \leftarrow \text{LLM}(\ell, R_3, k)$  {sys:  $\_\text{TOOL\_SYS}$ ;  $x_1$ : +groups, +next, +caps, +useCases per candidate}
21: refused  $\leftarrow$  "[]" in response or (parsed but no valid ID)
22: return  $C$  {score: null; refused tracked in trace}
```

- **Stage 1 — Namespace:** An ORD ID has the form $\langle \text{namespace} \rangle : \langle \text{type} \rangle : \langle \text{localId} \rangle : \langle \text{version} \rangle$, and the namespace is the coarsest boundary in the landscape, identifying the owning system (e.g. `sap.sf` for SuccessFactors, `corp.itsm` for IT service management). Narrowing by namespace first reduces the pool from 273 to ~ 54 resources (2 namespaces $\times \sim 27$ resources on average) with minimal LLM input. The system prompt $_\text{NS_SYS}$ instructs the LLM to select up to $\text{MAX_NAMESPACES} = 2$. It receives a per-namespace summary built by `_ns_summary()`,

Algorithm 4 Graph Walk — graph-walk traversal (`src/methods/method_c.py`)

Require: label ℓ , landscape R , result size k , prior resolved P_{prev}

Ensure: ranked list of $\leq k$ candidates, or $\emptyset$ if refused

```
1: idx  $\leftarrow$  Index( $R$ ) {builds et2res, grp2res, lob2res, next_out, calls_out}
2: ets, grps, lobs  $\leftarrow$  AllAnchors(idx)
3:  $A \leftarrow$  LLM( $\ell$ , ets, grps, lobs) {sys: _ANCHOR_SYS; up to 4 ETs, 3 groups, 2 LoBs}
4: initialise  $s_r \leftarrow 0$  for all  $r$ 
5: for et  $\in A.\text{entity\_types}$  do
6:    $s_r \mathrel{+}= 1.0$  for each  $r \in \text{et2res}[\text{et}]$  {ET hit}
7: end for
8: for  $g \in A.\text{business\_groups}$  do
9:    $s_r \mathrel{+}= 2.0$  for each  $r \in \text{grp2res}[g]$  {group hit}
10: end for
11: for  $l \in A.\text{lines\_of\_business}$  do
12:    $s_r \mathrel{+}= 0.5$  for each  $r \in \text{lob2res}[l]$  {LoB soft signal}
13: end for
14: seeds  $\leftarrow \{r \mid s_r > 0\} \cup P_{\text{prev}}$  {previous step as additional frontier seed}
15: for hop  $h = 1, \dots, \text{MAX\_HOPS}$  do
16:   decay  $\leftarrow \text{HOP\_DECAY}^h$ 
17:   for all  $u$  in frontier do
18:      $s_v \mathrel{+}= w(\text{co\_exposes}) \cdot \text{decay}$  for shared-ET neighbours  $v$ 
19:      $s_v \mathrel{+}= w(\text{co\_partOf}) \cdot \text{decay}$  for shared-group neighbours  $v$ 
20:      $s_v \mathrel{+}= w(\text{calls}) \cdot \text{decay}$  for integration-dep. targets  $v$ 
21:      $s_v \mathrel{+}= w(\text{processNext}) \cdot \text{decay}$  for process-next targets  $v$ 
22:   end for
23:   frontier  $\leftarrow$  newly reached nodes
24: end for
25:  $P \leftarrow$  top-RERANK_POOL resources by  $s_r$ 
26:  $C \leftarrow$  LLM( $\ell$ ,  $P$ ,  $k$ ) {sys: _RERANK_SYS; fallback to walk order on parse fail}
27: return  $C$ 
```

containing the resource count, top-3 entity types by frequency, and up to four lines of business. If Stage 1 returns no valid namespace, the method returns an empty result immediately.

- **Stage 2 — Modality:** With the namespace established, modality is a binary structural property derivable from the ORD resource type alone. `agent` and `apiResource` map to `operational` (live invocation surfaces); `dataProduct` and `eventResource` map to `analytical` (read-only or asynchronous signals). The system prompt `_MOD_SYS` explains the semantic distinction and biases toward `operational`, since most enterprise process activities drive live execution. If the LLM returns no valid modality, all modalities present in the pool are kept as a fallback.
- **Stage 3 — Entity Types:** Only after two structural filters does the method require semantic knowledge about which business objects the activity concerns. The system prompt `_ET_SYS` shows only the entity types actually present in the surviving pool with their frequency counts, requesting up to 3. Showing only reachable entity types prevents the LLM from selecting types that would produce an empty result. If the LLM returns an empty list, the stage is a pass-through. If the filtered pool would be empty, the full pool is kept as a safety fallback. Stage 3 is skipped entirely when no entity type references exist in the pool, reducing the total call count to 3.
- **Stage 4 — Tool Selection:** The final stage presents the filtered candidate pool via system prompt `_TOOL_SYS`. The pool size depends on the preceding filters and is not bounded explicitly. Each candidate is shown with `ordId`, `title`, `shortDescription`, and entity types. On enriched ORD (x_1), truncated `partOfGroups` group IDs, `processNext` successor IDs, `capabilities` labels, and the first `useCases` entry are also included to provide process-context for disambiguation. Refusal is handled explicitly: if the response contains a JSON `[]` or if all parsed IDs fall outside the valid candidate set, the result is marked as refused. A parse failure is not treated as refusal.

Algorithm 3 gives the pseudocode. See App. C.1, Listing 4 for an example trace on a design-time activity.

B.3.4 Graph Walk

Graph Walk is implemented in `src/methods/method_c.py` and consists of three stages. At the start of each `retrieve()` call, the function `_index()` builds five adjacency maps from the resource list: `et2res` (entity type to resources), `grp2res` (business group to resources), `lob2res` (line of business to resources), `next_out` (`processNext` edges), and `calls_out` (integration dependency edges). The `calls_out` map requires an additional read of the

Algorithm 5 Agentic Tools — typed ORD tools, `agentic (src/methods/method_d.py)`

Require: label ℓ , landscape R , skill registry S , prior resolved P_{prev} , budget T_{max}

Ensure: single committed resource, or $\emptyset$ if refused

```
1:  $\text{idx} \leftarrow \text{BuildIndices}(R, S)$  {7 lookup maps + skill registry; disk-reads calls_out}
2:  $\mathcal{T} \leftarrow \{\text{vocab tools, filter tools, get\_resource, skill tools, pick\_resource, refuse}\}$ 
3:  $\text{user\_msg} \leftarrow \ell + (\text{if } P_{\text{prev}} \neq \emptyset: \text{hint to call resources\_next\_after})$ 
4:  $\mathcal{C} \leftarrow [\{\text{sys: \_SYSTEM}\}, \{\text{user: user\_msg}\}]$ 
5: for step  $t = 1, \dots, T_{\text{max}}$  do
6:    $\text{choice} \leftarrow \text{"auto" if } t < T_{\text{max}} \text{ else "none"}$  {force text stop at budget limit}
7:    $(\alpha, \text{args}) \leftarrow \text{LLM}(\mathcal{C}, \mathcal{T}, \text{tool\_choice} = \text{choice})$ 
8:   if  $\alpha = \text{pick\_resource}$  then
9:     return  $[\text{args.ord\_id}]$  {committed; score: null}
10:  else if  $\alpha = \text{refuse}$  then
11:    return  $\emptyset$ 
12:  else if no tool call then
13:    break {agent stopped without terminal action}
14:  else
15:     $\text{result} \leftarrow \text{idx}.\alpha(\text{args})$  {compact summary for list/filter; full record for get_resource}
16:    append (assistant + tool result) to  $\mathcal{C}$ 
17:  end if
18: end for
19: return  $\emptyset$  {budget exhausted}
```

`ord_enriched.json` files from disk, because integration dependencies are stored at document level and are not part of the flat resource list passed by the caller.

- **Stage 1 — Anchor Inference:** The system prompt `_A_NCHOR_SYS` instructs the LLM to identify entity types, business process groups, and lines of business from the activity label that will serve as entry points into the graph. The user prompt `_anchor_prompt` shows only the vocabularies that actually exist in the landscape index, so the LLM can only select anchors from which the walk can depart. The instruction to return an empty list when unsure suppresses hallucinated anchors. Business process groups are populated from the `partOfGroups` semantic enrichment field, so on enriched ORD the anchor vocabulary is richer and the walk starts from more precise entry points.
- **Stage 2 — Seed and Walk:** Anchor matches seed the graph with differentiated initial scores: entity-type matches receive the highest weight, group matches a medium weight, and line-of-business matches a soft signal. If a `previous_resolved` resource is provided from the preceding plan step, it is added to the frontier as an additional seed, propagating process-sequential context into the walk. The walk then expands over `MAX_HOPS = 2` hops through four typed edge kinds, each discounted per hop. Explicit integration and sequential links (`calls`, `processNext`) carry higher weights than co-occurrence edges (`co_exposes`, `co_partOf`), reflecting that direct invocation and validated process sequence are stronger semantic signals.
- **Stage 3 — Re-rank:** The top `RERANK_POOL = 8` walk results are passed to a second LLM call. The system prompt `_RERANK_SYS` instructs the re-ranker to prefer agents for activities that describe work being done, APIs for CRUD operations on master data, and data products for analytical reads. Each candidate is shown with `ordId`, `type`, `title`, `capabilities`, `useCases`, and `shortDescription` only, without full free-text descriptions, because the graph walk has already established topical relevance. Refusal follows the same convention as Progressive Disclosure: an explicit `[]` in the response marks refusal, while a parse failure falls back to the walk ordering rather than refusing.

Algorithm 4 gives the pseudocode. See App. C.1, Listing 4 for an example trace on a design-time activity.

B.3.5 Agentic Tools

Agentic Tools is implemented in `src/methods/method_d.py`. Unlike Embedding Cosine, Progressive Disclosure, and Graph Walk, which follow fixed retrieval pipelines, Agentic Tools gives an LLM agent a typed tool surface over the ORD landscape and lets it compose its own retrieval strategy. The agent is constrained to at most `MAX_TOOL_CALLS = 8` tool calls before a forced stop.

- **Index construction:** At the start of each `retrieve()` call, `_build_indices()` constructs lookup structures over the resource list covering namespace, entity type, line of business, business group, capability, and integration dependency targets per namespace. On enriched ORD, additional fields become available: `processNext` edges, `partOfGroups` business groups, and `capabilities` are populated by the design-time flow and extend the index

Algorithm 6 Agentic Raw — raw file access, `agentic (src/methods/method_e.py)`

Require: label ℓ , prior resolved P_{prev} , budget $T_{\text{max}}=10$, line cap $L_{\text{max}}=200$

Ensure: single committed resource, or $\emptyset$ if refused

```
1: {resources, top_k, skills ignored — Agentic Raw navigates the filesystem directly}
2:  $\nu \leftarrow \text{LoadNotes}()$  {pre-loaded into system prompt; no tool call needed}
3:  $\mathcal{T} \leftarrow \{\text{list\_dir}, \text{read\_file} \text{ (capped at } L_{\text{max}}), \text{read\_notes}, \text{write\_notes}, \text{pick\_resource}, \text{refuse}\}$ 
4:  $\mathcal{C} \leftarrow [\{\text{sys: } \_SYSTEM\_BASE + \nu\}, \{\text{user: } \ell + P_{\text{prev}} \text{ hint}\}]$ 
5: for step  $t = 1, \dots, T_{\text{max}}$  do
6:   choice  $\leftarrow$  "auto" if  $t < T_{\text{max}}$  else "none"
7:    $(\alpha, \text{args}) \leftarrow \text{LLM}(\mathcal{C}, \mathcal{T}, \text{tool\_choice}=\text{choice})$ 
8:   if  $\alpha = \text{pick\_resource}$  then
9:     return  $[\text{args.ord\_id}]$ 
10:  else if  $\alpha = \text{refuse}$  then
11:    return  $\emptyset$ 
12:  else if no tool call then
13:    break
14:  else if  $\alpha = \text{write\_notes}$  then
15:     $\text{SaveNotes}(\text{args.content}[: 4096])$  {persists across calls; capped at 4KB}
16:    append tool result to  $\mathcal{C}$ 
17:  else
18:    result  $\leftarrow \alpha(\text{args})$  {list_dir: entries; read_file: content truncated at  $L_{\text{max}}$ }
19:    append result to  $\mathcal{C}$ 
20:  end if
21: end for
22: return  $\emptyset$  {budget exhausted}
```

with process-context that is absent on clean ORD. The skill registry is also indexed here. In Skill-Adjusted mode, the upstream Intent Resolver has already matched a skill and passes its identifier to Agentic Tools, which can then call `describe_skill` to read the `ord_confirmed` resources of adjacent steps as retrieval context.

- **Tool surface:** The 16 tools are grouped into five categories. Vocabulary tools (`list_namespaces`, `list_entity_types`, `list_lines_of_business`, `list_business_groups`, `list_capabilities`) return flat lists from the indices. Filter tools (`resources_exposing`, `resources_in_lob`, `resources_in_group`, `resources_with_capability`) return compact summaries with `ordId`, `type`, `title`, and `shortDescription` only, so results stay small enough to fit in context without overwhelming the agent. Graph tools (`integration_dependencies`, `resources_next_after`) expose the two cross-resource edge kinds (integration dependencies per namespace and `processNext` successors of a given `ordId`). The detail tool `get_resource` returns the full typed descriptor for a single resource and is intended to be called only after the pool has been narrowed by the filter or graph tools. The skill tools (`list_skills`, `describe_skill`) expose the pre-bound process registry. The two terminal tools `pick_resource` and `refuse` must end every session. `pick_resource` commits to one primary resource; if `top_k > 1`, additional candidates are filled from `ordIds` encountered in prior tool results. Because refusal is an explicit tool call rather than an empty array, Agentic Tools avoids the parse-failure ambiguity present in Progressive Disclosure and Graph Walk.
- **System prompt:** The system prompt `_SYSTEM` is structured in three blocks. The ORD grounding block explains the typed fields the agent can filter on, including which fields are only present on enriched ORD. The architecture context block informs the agent that mode and skill have already been resolved upstream, and that a provided skill ID is a strong signal worth inspecting with `describe_skill`. The tool usage guidance block suggests a productive search pattern: start with vocabulary tools to understand what filters are available, narrow the pool by intersecting filter results, inspect survivors with `get_resource`, and stop as soon as one candidate clearly dominates. This guidance reduces unnecessary tool calls without constraining the agent to a fixed sequence.
- **Conversation loop:** Each step issues one LLM call with the full message history appended, so the agent can reason over its own prior tool results. The call uses `tool_choice="auto"` for all steps except the last, where it switches to "none" to force a text termination if the budget is exhausted without a terminal action. If `previous_resolved` resources are available, they are inserted into the user message with an explicit instruction to consider `resources_next_after`, so plan-sequential context reaches the agent without requiring an additional tool call.

Algorithm 5 gives the pseudocode. See App. C.1, Listing 4 for an example trace on a design-time activity and App. C.3, Listing 6 for the enrichment effect on a runtime case.

Algorithm 7 Agentic Hybrid — Agentic meta-retrieval (`src/methods/method_f.py`)

Require: label ℓ , resources R , top- k , use_graph, allow_refuse

Ensure: ranked candidate list of length top- k , or $\emptyset$ if refused

```
1:  $T \leftarrow$  tool set {embedding_search, progressive_filter, graph_walk, resources_with_capability,
   resources_exposing, get_resource, pick_resource, refuse}
2:  $M \leftarrow$  [system prompt, user prompt( $\ell$ , use_graph)]
3:  $V \leftarrow \square$  {ordered list of visited candidates}
4: for  $s = 0$  to MAX_TOOL_CALLS do
5:    $(t_c, m) \leftarrow$  LLM( $M$ , tools= $T$ )
6:   if  $t_c$  is empty then
7:     break {model returned final text}
8:   end if
9:   for each call  $c \in t_c$  do
10:    if  $c.name = \text{pick\_resource}$  then
11:      picked  $\leftarrow c.args.ord\_id$ ; break outer loop
12:    else if  $c.name = \text{refuse}$  then
13:      return  $\emptyset$ 
14:    else
15:      result  $\leftarrow$  ExecuteTool( $c.name$ ,  $c.args$ ,  $R$ )
16:      append returned ordIds to  $V$  (skipping duplicates)
17:      append  $c$  and result to  $M$ 
18:    end if
19:  end for
20: end for
21: return [picked] + ( $V \setminus \{\text{picked}\}$ ): top- $k-1$ ]
22: {if no commit: fallback  $\leftarrow$  method_a.retrieve( $\ell$ ,  $R$ ,  $k$ )}
```

B.3.6 Agentic Raw

Agentic Raw is implemented in `src/methods/method_e.py` and is the most unconstrained retrieval design. Where Agentic Tools gives the agent a typed ORD tool surface, Agentic Raw gives it only directory listing and file reading. The research question is whether a capable LLM needs a typed abstraction layer at all, or whether raw file access is sufficient.

- **Tool surface:** R has six tools. `list_dir(path)` and `read_file(path)` navigate the `benchmark/landscape/` directory tree. `read_notes()` and `write_notes(content)` provide a persistent scratch pad. `pick_resource` and `refuse` are the same terminal actions as in T . There are no typed filter or vocabulary tools; the agent must infer structure from the raw JSON it reads. Read access is sandboxed to the `benchmark/` subtree, preventing directory traversal.
- **Read cap:** `read_file` is capped at 200 lines per call. The cap carries no ORD schema knowledge and does not truncate at array boundaries or skip irrelevant fields. This preserves the R -vs- T contrast: any schema-aware truncation would implicitly provide the typed structure that distinguishes T from R .
- **Persistent notes:** The notes file survives across `retrieve()` calls within a benchmark run. At the start of each call the current notes are injected into the system prompt, saving one tool call. The agent is instructed to update its notes after each query so later queries can skip re-discovery. Before each benchmark run `reset_notes()` wipes the file to ensure a clean starting state.
- **System prompt:** `_SYSTEM_BASE` uses the same structure as Agentic Tools but omits the architecture context block because Agentic Raw has no access to planner state. The budget is set to 10 tool calls, higher than Agentic Tools' 8, because each call is a raw file read rather than a typed query.
- **Ignored parameters:** `resources`, `top_k`, and `skills` are accepted for signature compatibility but ignored. Agentic Raw navigates the filesystem directly and does not use the in-memory resource list by design.

Algorithm 6 gives the pseudocode. See App. C.1, Listing 4 for an example trace on a design-time activity and App. C.5, Listing 8 for the refusal trajectory on an out-of-scope request.

B.3.7 Agentic Hybrid

Agentic Hybrid is implemented in `src/methods/method_f.py`. It is an agentic composite that combines Embedding Cosine, Progressive Disclosure, and Agentic Tools' retrieval mechanisms as tools, allowing an LLM agent to select and compose them per query. Its purpose is to test whether adaptive composition can exceed each individual method on $R@1$ without proportionally increasing token cost.

F does not follow a fixed pipeline. It gives a reasoning LLM a tool surface of eight functions and lets the agent decide which combination to invoke:

- **embedding_search(query, top_k)** wraps Embedding Cosine’s cosine similarity over title, shortDescription, and description (plus enrichment fields on Enriched-ORD). Best for broad semantic recall.
- **progressive_filter(query, top_k)** wraps Progressive Disclosure’s four-stage pipeline (namespace → modality → entity type → tool selection). Effective when the target namespace or entity type can be inferred from the query.
- **graph_walk(anchor_ord_ids, top_k)** wraps Graph Walk’s multi-hop walk over processNext, co_exposes, calls, and co_partOf edges from a set of anchor resources. Enriched-ORD only.
- **resources_with_capability(capability)** lists resources whose capabilities array contains a given verb-noun label. Enriched-ORD only.
- **resources_exposing(entity_type)** lists resources exposing a given ODM entity type.
- **get_resource(ord_id)** returns the full descriptor of one resource for inspection before committing.
- **pick_resource(ord_id, reason)** is the terminal commit call. F’s result is the ordId passed here.
- **refuse(reason)** is available at runtime (allow_refuse=True). Not available at design-time.

The system prompt instructs the agent to start with embedding search for broad recall, then cross-check with a different approach (progressive filter or capability lookup) when results are close, and to inspect candidates with `get_resource` before committing. On Enriched-ORD it also advises checking capabilities and graph neighbours. The agent is bounded by `MAX_TOOL_CALLS = 10`. If the agent stops without a commit, Embedding Cosine is called as a fallback. The `skills` and `previous_resolved_ord_ids` parameters are accepted for signature compatibility but ignored. The final candidate list is `[picked_id]` followed by other resources the agent encountered during tool calls, in order of first appearance.

This design affects R@5. Embedding Cosine returns a ranked list of five candidates, so the correct resource often appears somewhere in it even if not at rank 1. Agentic Hybrid commits to exactly one resource and builds the remaining slots from whatever the agent looked at during its tool calls. If the agent picks the right resource but looked at different candidates on the way, the ground-truth resource may not appear in those remaining slots at all. In numbers: Agentic Hybrid reaches R@1 = 45.0% on Dynamic Enriched, higher than Embedding Cosine’s 28.7%, but its R@5 drops to 53.7% against Embedding Cosine’s 70.0%. Agentic Hybrid finds the right answer more often at rank 1, but covers fewer of the right answers within the top five.

Algorithm 7 gives the pseudocode. See App. C.1, Listing 4 for the design-time behaviour alongside the other methods, and App. C.4, Listing 7 for an end-to-end pipeline trace on a multi-intent runtime case.

Listing 3: ORD enrichment example: `my.mes:agent:ProductionChangeOrchestrator:v1` in its clean state (x_0 , left) and enriched state (x_1 , right). Fields absent in x_0 are marked – absent –; the four added fields are written back after human validation.

```

1  -- CLEAN (systems/my.mes/ord.json) --          -- ENRICHED (systems_enriched/my.mes/ord_enriched.json) --
2
3  "ordId": "my.mes:agent:ProductionChangeOrchestrator:v1"
4  "title": "Production Change Orchestrator"
5  "version": "1.0.0"
6  "entityTypes": [                                (unchanged)
7    "sap.odm:entityType:EngineeringChange:v1",
8    "sap.odm:entityType:Material:v1",
9    "sap.odm:entityType:ProductItem:v1"
10 ]
11
12 "capabilities": -- absent --                      "capabilities": [
13                                                     "manage-material-updates",
14                                                     "validate-engineering-specifications",
15                                                     "sequence-work-orders",
16                                                     ... (10 total)
17 ]
18
19 "partOfGroups": -- absent --                      "partOfGroups": [
20   { "groupId": "proc_ManufacturingChangeCompliance_v1",
21     "groupTypeId": "bpmmn"},
22   ... (3 total)
23 ]
24
25 "processNext": -- absent --                      "processNext": [
26   "sap.sf:apiResource:CompensationAndSalaryManagement:v1"
27 ]
28
29 "useCases": -- absent --                          "useCases": [
30   "Operations manager submits engineering change
31   order for new product variant ...",
32   ... (8 total)
33 ]

```

B.4 Design-Time Data Flow

The design-time pipeline is implemented in `src/design_time/` and consists of two modules. `dt_cases.py` builds the test cases from the process model files and the human-validated annotations. `dt_benchmark.py` runs the matching experiments across all seven retrieval methods on Clean-ORD only, collects metrics, and writes results to `results/design-time/`. Enriched-ORD is not evaluated at design-time because semantic enrichment is the output of this flow, not an input.

Case construction. `dt_cases.py` loads the activities from `activity_cases.json` in the generation pipeline's output directory (`benchmark/test_cases/design_time/output/`), with one entry per process step and its ground-truth resource. Each activity label is prefixed with the human-readable process title (parsed from the `proc_*.xml` `id` attribute in the same directory's `processes/` subfolder) so methods receive the same process context an orchestrator would have at execution time. Each case is a dict with fields including `case_id`, `process`, `label` (the process-prefixed activity), `label_raw` (the activity alone), `expected_ordIds`, and an `is_gt` flag marking ground-truth-eligible steps.

Benchmark runner. `dt_benchmark.py` iterates over all seven retrieval methods (Baseline, Embedding, Progressive, Graph, Tools, Raw, Hybrid) on Clean-ORD. For each case it calls `method.retrieve(label, resources)` and scores the result against the expected resource IDs. The metrics computed per case are `R@1` (Top-1 candidate is in the expected set) and `Recall@5` (at least one expected ID appears in the top five). For out-of-scope cases where `expected_ordIds` is empty, `R@1` is not computed. Instead the scorer checks whether the method correctly returned no candidate (`correctly_refused`). Every case writes a full trace JSON to `results/design-time/traces/<method>/<case>.json`, and aggregate results are written to `results/design-time/summary.json` and `summary.csv`. A representative design-time trace contrasting all seven methods on the same activity is shown in App. C.1 (Listing 4).

Output. The design-time data flow produces two artefacts: the enriched ORD documents (`benchmark/landscape/systems_enriched/<ns>/ord_enriched.json`) and the skill registry (`benchmark/test_cases/design_time/output/skills/`). Construction logs are written to `benchmark/test_cases/design_time/logs/` (`process_construction_log.json`, `skill_construction_log.json`, `enrichment_log.json`). Listing 3 shows an enriched ORD example, and App. B.2 (Listing 2) a full skill artefact.

Table 12: Mode-routing accuracy of the runtime pipeline (Intent Resolver plus Planner), per orchestration mode. The weighted average is 70.0%. Skill-Guided routing is the only decision it makes, since its resources are pinned in the matched skill.

Mode	<i>n</i>	Routing accuracy (%)
Skill-Guided (SG)	30	70.0
Skill-Adjusted (SA)	20	90.0
Dynamic (DY)	40	60.0
Weighted avg.	Σ90	70.0

B.5 Runtime Data Flow

The runtime pipeline is implemented in `src/runtime/` and consists of three modules that correspond to the pipeline stages.

Intent resolver. `intent_resolver.py` maps a natural-language user request to the closest registered skill. It sends only the skill IDs and short descriptions to the LLM, not the step lists, because at this stage the question is only whether a skill exists for the intent. Keeping step inspection out of this call reduces token cost and separates the two decisions (does a skill exist vs. does it cover the request). The LLM returns a `best_skill_id` and a one-sentence reason, or `null` if no skill matches.

Planner. `planner.py` takes the matched skill and the user request and, in a single LLM call, decides coverage together with any gap step labels. The full step list of the matched skill is sent so the model can compare steps against the activities implied by the request. The response carries a coverage label (`full` or `partial`) and, when `partial`, an array of concrete gap step labels. Coverage `full` maps to Skill-Guided, coverage `partial` with a non-empty gap list maps to Skill-Adjusted, and no skill match falls through to Dynamic where the user request itself becomes the sole step. Each gap step label is then passed to the same retrieval method that runs at design time, so the gap resolution path is identical to the design-time matching path.

Benchmark runner. `rt_benchmark.py` runs the 7×2 factorial experiment (seven retrieval methods, two ORD states) over the runtime test cases, which are split into four JSON files by expected mode under `benchmark/test_cases/runtime/output/`: `skill_guided.json`, `skill_adjusted.json`, `dynamic.json`, and `out_of_scope.json`. For each case it calls `rt_planner.plan_and_resolve()`, which runs the full pipeline and returns a plan with per-step candidates and traces. The metrics computed per case are `top1_acc`, `acceptable`, `candidate_recall`, `mode_routing_ok`, `gap_detection` (Skill-Adjusted), and `step_coverage` (Skill-Guided).

Output. Results are written to `results/runtime/<mode>/` with summary CSVs and a full per-case trace JSON recording the resolved mode, the per-step candidates, and the method-specific decision trace. Representative traces covering all four modes are shown in App. C: Skill-Guided (C.4.2), Dynamic with Agentic Hybrid (C.4.3), Skill-Adjusted enrichment effect (C.4.4), and Out-of-Scope refusal (C.4.5).

Routing accuracy per mode. Tab. 12 reports the mode-routing accuracy of the Intent Resolver and Planner, which is the number quoted in Sec. 4. Skill-Adjusted routes most reliably because its partial-match cases carry strong disambiguation signals; Dynamic is hardest because a genuine no-skill case is difficult to separate from a missed skill match.

B.6 Requirements Coverage

This appendix maps the ten requirements R1–R10 onto the architectural elements of the design chapter. Each requirement is carried by a specific layer or data flow.

DO1 (Process Execution) is carried by the runtime modes spanning the structuredness continuum and by the skill registry in the Semantic Layer. *R1 (full spectrum)* is realised by the three orchestration modes: Skill-Guided for fully modelled processes, Skill-Adjusted for partial coverage, Dynamic for unmodelled requests. *R2 (modular, dynamically loadable skills)* is realised by treating each skill as a markdown artefact that is loaded into the prompt only when selected. *R3 (agentic execution of the BPM implementation phase)* is realised by the runtime pipeline that turns natural-language input into an executable plan over ORD resources.

DO2 (Description Layer) is carried by ORD as the self-describing registry with internal structure. *R4 (registry with typed structure)* is met by ORD’s per-system Service Provider Interface and its typed field groups (Resource Types, Entities & Domains, Relations). *R5 (vendor-neutral interfaces)* is met by the open governance under Linux Foundation Europe / NeoNephos and the reuse of OpenAPI under ORD.

DO3 (Semantic Enrichment) is carried by the design-time data flow that persists semantic content into ORD. *R6 (business*

meaning beyond syntax) is met by `entityType`, `authority namespaces`, `integrationDependency`, and the four DT-enriched fields that together lift ORD descriptions from invocation-level syntax to business-level meaning. *R7 (lightweight, harvestable, SBPM without ontology stack)* is met by writing back these fields through ORD’s label and extension-field mechanism (no schema fork) and by harvesting the source content from existing BPMN/CMMN artefacts. *R8 (typed process—IT link in the description layer itself)* is met by the four DT-enriched fields (`partOfGroups`, `processNext`, `capabilities`, `useCases`) written back into ORD by the design-time flow.

DO4 (Orchestration Layer) is carried by the Semantic Layer’s retrieval methods and by the trace-emitting Orchestration Layer. *R9 (disambiguation in dense action spaces)* is met by Embedding, Progressive, Graph, Agentic Tools, Agentic Raw, and Agentic Hybrid operating over the same ORD landscape at both design-time and runtime. *R10 (robustness against description smells at the Semantic Layer)* is met by the per-method trace that captures cosine scores, namespace and tool completions, traversed graph edges, agent tool histories, and refusal flags, and by the ability of Graph, Agentic Tools, and Agentic Raw to explicitly refuse when no resource matches the request.

C. Representative Retrieval Traces

This section applies the data flows of App. B.4 (design-time) and App. B.5 (runtime) to the ORD-Bench test cases and inspects what they produce. Full per-case traces for all 240 design-time cases and all 110 runtime cases are stored under `results/design-time/traces/<method>/` and `results/runtime/<mode>/traces/<condition>/`. Each trace records the complete method execution. This section shows a small set of representative cases that together expose every mechanism in the pipeline. The design-time case `proc_002_s01` contrasts all seven retrieval methods on the same activity. The four runtime cases add mechanisms that a design-time activity cannot expose: skill-based routing, the enrichment effect, multi-intent decomposition, and refusal.

Condition labels and reported numbers. A runtime condition is written `<method><state>` with `state 0` for Clean-ORD and `1` for Enriched-ORD (e.g. `A0`, `F1`), giving the 7×2 factorial. Three suffixed variants isolate a retrieval measurement from an upstream routing decision, so a method is scored on retrieval quality rather than on router or decomposer errors. A *forced* run (suffix `f`) re-runs a case in its correct mode when the Intent Resolver misrouted it. A *gap-forced* run (suffix `gf`) injects the ground-truth gap-step labels so gap retrieval is measured without depending on the Planner. A *multi-hint* run (suffix `mh`) executes each sub-query of a multi-intent case as an independent retrieval call, scored per sub-query with partial credit. Which of these feeds the reported number differs per mode.

Skill-Guided is scored by routing accuracy alone (`src/runtime/eval_sg_routing.py`): the Intent Resolver picks a skill from the prompt and the skill descriptions, so the ORD state is irrelevant and only the plain per-case record `sg-NN.json` is kept, with no retrieval condition. *Skill-Adjusted* reports per-method retrieval over the gap steps: the `gf` record is preferred over the plain one wherever it exists, because it runs gap retrieval on the ground-truth gap labels rather than on what the Planner produced, and each `gf` record is merged back onto its base condition `<method><state>`. *Dynamic* numbers (the per-mode results (Tab. 4)) are *combined*: for every method the single-intent half comes from the forced records `<method><state>f` over the 20 single-intent cases, and the multi-intent half from the multi-hint records `<method><state>mh` over the 20 multi-intent cases, merged as $(\text{single} \times 20 + \text{multi} \times 20)/40$ by `src/runtime/compute_combined_summary.py`. *Out-of-Scope* uses the plain condition directly, since abstention has no ground-truth resource to force retrieval towards. Every runtime figure reported in the main paper (Sec. 4) is computed exactly this way, one scoring rule per mode.

C.1 Design-Time — `proc_002_s01`

The activity label is “*ProductLifecycleCompliance process: Analyze product cost and complexity impact, Evaluate newly proposed materials and product designs against cost and complexity benchmarks.*” The ground truth resource lives in the PLM namespace, while a HIGH-tier distractor with the same entity types and a related line of business sits in a different namespace. Disambiguation therefore requires either landscape-level system knowledge or a structural signal that separates the two candidates. Listing 4 shows how each retrieval method handles this activity in turn, in a single stacked trace so the mechanisms can be read side by side.

C.2 Runtime — Skill-Guided routing on `sg-03`

Skill-Guided cases exercise routing rather than retrieval. The only metric is whether the Intent Resolver picks the correct skill from the registry; the matched skill’s steps then carry `ord_confirmed` annotations that are read directly, without a scored retrieval step. `sg-03` is representative of the 21 correctly-routed cases (70% routing accuracy). Listing 5 shows the routing decision that is scored.

Listing 4: All retrieval methods on `proc_002_s01`. Reading top to bottom shows the progression from free text (Baseline, Embedding) through progressive structure (Progressive), graph walk (Graph), typed tools (Tools), file-level agent (Raw), to the hybrid composite (Hybrid).

```

1 Activity: "ProductLifecycleCompliance process:
2   Analyze product cost and complexity impact ..."
3 Ground truth: siemens.plm dataProduct on ProductItem+Material
4 Distractor:  sap.s4 dataProduct, same entity types, related LoB
5
6 [B] Free-text baseline
7   single LLM call over 273 (ordId, title, shortDescription) lines
8   GT title contains the activity's key phrase verbatim
9   -> picks GT              13,646 tokens      OK
10
11 [E] Embedding cosine
12   embed(query) + embed(273 resources) [one-time landscape embed]
13   cosine top-5 ranks GT first
14   -> picks GT          29,295 tokens full / ~185 amortised      OK
15
16 [P] Progressive disclosure (staged LLM)
17   Stage 1 namespace filter drops the GT namespace early
18   Stages 2-4 operate on the wrong candidate pool
19   -> picks distractor          2,512 tokens      wrong
20
21 [G] Graph walk from ET/LoB anchors
22   seeded from correct entity types and lines of business
23   walk visits many reachable resources across namespaces
24   LLM reranker holds GT at rank 1
25   -> picks GT              4,019 tokens      OK
26
27 [T] Typed ORD tools (8 tool calls)
28   list_entity_types / list_lines_of_business / list_capabilities
29   resources_in_lob("Product Lifecycle Management")
30   resources_with_capability("analyze-product-costs")
31   resources_with_capability("evaluate-complexity")
32   get_resource(candidate)
33   pick_resource
34   -> picks GT              24,405 tokens      OK
35
36 [R] Raw file agent (10 tool calls)
37   read_notes -> list_dir -> 6x read_file across systems_enriched
38   write_notes with a landscape inventory
39   no internal tiebreaker between same-namespace candidates
40   -> picks a wrong resource in the same namespace
41   94,040 tokens      wrong
42
43 [H] Agentic meta-retrieval (5 tool calls)
44   embedding_search(activity label) -> top-10 candidates
45   3x get_resource on top hits (GT + 2 distractors)
46   pick_resource(GT)
47   -> picks GT              7,915 tokens      OK

```

Listing 5: Routing trace for `sg-03` (`results/runtime/skill_guided/routing/sg-03.json`). Skill-Guided is scored on this decision alone.

```

1 User request:
2   "We are launching a new product line -- coordinate everything
3   from receiving materials through engineering readiness,
4   documentation, and operational safety before production starts."
5
6 [Intent Resolver]
7   input: user request + 30 skill descriptions (id + short desc)
8   1 LLM call, 1,656 tokens
9   skill_picked = proc_012      (expected: proc_012)
10  reason: "This skill orchestrates comprehensive preparation and
11  launch of a new product line across procurement, engineering,
12  operations, and sales ..."
13  routing_ok = 1
14
15 Case scored: routing-correct. The SG metric is routing accuracy
16 only; no Planner or retrieval step is evaluated for this mode.

```

C.3 Runtime — Skill-adjusted on sa-16

In the case shown here, the Planner extracts three gap steps from a production-schedule request, against two ground-truth gap resources (one on Compensation, one on Payroll). Every method runs twice, on Clean-ORD (condition suffix 0) and on Enriched-ORD (suffix 1), so the four enrichment fields are the only difference between the two runs. Agentic Tools resolves *one* of the three gaps on Clean-ORD and *two* on Enriched-ORD. The compensation gap resolves on both states because the resource title overlaps the request wording. The difference is the payroll gap: on Enriched-ORD the capability filter

Listing 6: Agentic Tools on the payroll gap of sa-16 (gap 3 of 3). The process-payroll capability filter that Enriched-ORD provides is the decisive step; Clean-ORD has no equivalent signal and falls back to a broad line-of-business filter.

```

1 Case sa-16: 3 gap steps, 2 ground-truth gap resources
2   gap 1 "update employee compensation ..." -> resolved on BOTH states
3   gap 2 "process merit adjustments"          -> unresolved on both
4   gap 3 "coordinate payroll updates"         -> only Enriched resolves
5 Clean-ORD (T0): 1/3 gaps    Enriched-ORD (T1): 2/3 gaps
6
7 --- gap 3: "Coordinate financial/payroll system updates" ---
8 Ground truth: workday.hcm:apiResource:PayrollProcessingDataManagement
9
10 [Enriched-ORD (T1)]
11   resources_next_after(compensation agent)
12   list_capabilities()    list_lines_of_business()
13   resources_in_lob("Finance")          -> broad finance pool
14   resources_with_capability(
15     "process-payroll")                -> workday.hcm payroll
16   get_resource(workday.hcm payroll)
17   get_resource(sap.s4 payroll)
18   pick_resource(workday.hcm payroll)   -> GT resolved
19
20 [Clean-ORD (T0)]
21   resources_next_after(compensation agent)
22   list_lines_of_business()
23   resources_in_lob("Finance")          -> broad finance pool
24   list_capabilities()                -> empty on Clean-ORD
25   get_resource(sap.s4 payroll)
26   get_resource(sap.ariba procurement)
27   pick_resource(sap.s4 payroll)        -> wrong pick

```

Listing 7: End-to-end pipeline trace for dy-21 with Agentic Hybrid on Enriched-ORD. Routing produces the inputs that retrieval consumes.

```

1 User request:
2   "We need to analyze the workforce skills and engineering
3   requirements for our upcoming product line transition,
4   while simultaneously ensuring our compensation structures
5   are competitive and properly aligned with the new roles
6   and responsibilities our team will take on during this shift."
7
8 [Intent Resolver + Planner]
9   force_mode = dynamic (multi-hint run: routing bypassed so
10  every intent reaches retrieval regardless of routing errors)
11
12 [Intent Decomposer]
13   splits the request into two sub-queries:
14   SQ1: workforce skills inventory + engineering requirements
15   SQ2: review compensation structures for new roles
16   n_intents matches the ground truth split of 2
17
18 [Agentic Hybrid retrieval on SQ1] enriched-ORD
19   embedding_search(top_k=5)          -> 5 candidates, GT included
20   resources_with_capability("analyze-workforce-skills") -> 0
21   resources_with_capability("manage-engineering-requirements") -> 0
22   get_resource(GT)    get_resource(distractor)
23   pick_resource(sap.s4 EngineeringChangeImpactAndWorkforce...)
24   tokens: 8,030    top1: OK (GT resolved)
25
26 [Agentic Hybrid retrieval on SQ2] enriched-ORD
27   embedding_search(top_k=8)          -> 8 candidates, GT absent
28   resources_with_capability("manage-/evaluate-/review-
29   compensation")                    -> all 0
30   progressive_filter(top_k=5)        -> 4 candidates
31   get_resource x4 on compensation candidates
32   pick_resource(workday.hcm CompensationTrendAnalytics)
33   tokens: 12,811    top1: wrong
34   (GT sap.sf:apiResource:CompensationAndSalaryManagement
35   never surfaced; recall = 0)
36
37 [Aggregate]
38   per-sub-query top1 = {SQ1: 1, SQ2: 0}
39   case-level top1 = 1/2; SQ2 GT missed in top-5 entirely

```

process-payroll isolates the correct workday.hcm resource, whereas Clean-ORD has no capability field and falls back to a line-of-business filter that commits to the wrong sap.s4 payroll API. Listing 6 shows that side by side.

Listing 8: Agentic Raw refusal trace for oos-01 on Clean-ORD (11 tool calls, 74,602 tokens). On Enriched-ORD Agentic Raw refuses immediately in a single call.

```

1 User request:
2   "We need a way to automatically flag suppliers whose credit
3     ratings or financial health metrics have deteriorated in the
4     past 90 days so we can proactively assess whether to reduce
5     order volumes or increase payment terms ..."
6
7 [Agentic Raw on Clean-ORD]
8   list_dir(landscape)  list_dir(benchmark/landscape)
9   list_dir(benchmark/landscape/systems)  -> 10 systems
10  read_file(sap.ariba/ord_enriched.json)
11  read_file(sap.s4/ord_enriched.json)
12  list_dir(systems_enriched/sap.s4)
13  read_file(sap.odm/ord_enriched.json)
14  read_file(sap.ariba/ord.json)
15  write_notes("LANDSCAPE INVENTORY: ... no resource flags
16    supplier financial health; ariba = invoice/compliance,
17    s4 = procurement matching ...")
18  read_file(sap.s4/ord.json)
19  refuse("The landscape contains no resource for monitoring
20    supplier credit ratings or financial health deterioration;
21    available supplier resources focus on procurement,
22    compliance, and invoice matching.")
23
24 Case scored: correct refusal (most expensive refuser on Clean).

```

C.4 Runtime — Dynamic pipeline with Agentic Hybrid on dy-21

Dynamic cases cover ad-hoc requests with no matching pre-modelled skill, evaluated under both Clean-ORD (state 0) and Enriched-ORD (state 1). The multi-intent case shown here bundles two distinct business goals into one user request: one about workforce and engineering requirements for a product-line transition, the other about compensation alignment. The Intent Resolver must recognise that no single skill covers both, the Planner must split the utterance into two sub-queries, and the retrieval method must find one resource for each. Listing 7 shows one such end-to-end pipeline on Enriched-ORD.

C.5 Runtime — Refusal on oos-01

Refusal cannot be observed on any of the previous cases because they all have a valid ground truth. The Out-of-Scope case shown here asks for a supplier credit-rating capability that does not exist anywhere in the 273-resource landscape. On this case Embedding, Progressive, and Agentic Tools commit to procurement-adjacent picks. Graph refuses only under Enriched-ORD, where the richer capability edges make the missing capability observable. Agentic Raw refuses on both states, but the two runs differ sharply in effort: on Clean-ORD it reads several full `ord.json` files across systems before concluding (11 tool calls, 74,602 tokens), whereas on Enriched-ORD the capability fields let it refuse immediately (1 call, 3,084 tokens). Listing 8 shows the example.

D. Design-Time Results

Design-time evaluation matches process-activity labels to the one correct resource among all 273 candidates without semantic enrichment. It serves as both a retrieval benchmark and the mechanism that produces semantic enrichment: the typed fields written back into ground-truth resources at design time are exactly the fields used in runtime Enriched-ORD evaluation.

Table 13 reports R@1 and R@5 for all seven methods on the 240 design-time cases. Agentic Tools is the only single-strategy method to exceed the Baseline (R@1 = 26.7% vs. 24.6%), through typed entity-type and line-of-business tool calls that constrain the candidate pool before committing. Embedding Cosine ties the Baseline on R@1 but leads R@5 (57.9%): the correct resource appears in the top five in the large majority of cases, confirming the architecture reliably narrows the resource space even when it cannot rank it first. Graph and Agentic Raw are both dominated — Graph’s walk reaches 110+ candidates due to shared entity types, and Agentic Raw reads full ORD files of up to 94k tokens while committing too early to an adjacent resource. The Agentic Hybrid composite reaches R@1 = 51.3% and R@5 = 77.1% at 8.5k tokens per case, the best of all seven methods at design time.

Because the design-time set has $n=240$, its 95% bootstrap confidence intervals are markedly narrower than the runtime ones and several method differences separate cleanly. Agentic Hybrid (R@1 = 51.3%, 95% CI [45.8, 58.3]; R@5 = 77.1%, [71.7, 82.5]) sits entirely above every other method on both metrics, so its design-time advantage is statistically supported rather than directional. At the other end, Agentic Raw (R@1 = 6.7%, [3.8, 10.0]) and Graph (7.5%, [4.6, 11.2]) fall clearly below the Baseline (24.6%, [19.2, 30.0]) with non-overlapping intervals. This contrast with the wide runtime intervals illustrates the general point: the directional runtime findings would become statistically decisive at the larger sample sizes targeted in future work.

Table 13: Design-time results on 240 process-activity cases. R@1 = recall at rank 1; R@5 = recall within top 5; Tokens = average per case. Bold = best per column excluding baseline. *Agentic Hybrid is the agentic composite.

Method	R@1	R@5	Tokens
Baseline	24.6	57.5	~15k
Embedding	24.6	57.9	~185
Progressive	22.1	39.6	3.3k
Graph	7.5	10.8	2.2k
Tools	26.7	32.9	24k
Raw	6.7	6.7	35k
Hybrid*	51.3	77.1	8.5k

E. Dynamic Results: Single- vs. Multi-Intent

Table 14 decomposes the Dynamic mode ($n=40$) into its single-intent ($n=20$) and multi-intent ($n=20$) halves, alongside the combined figures reported in the main text (Sec. 4). Combined R@1 stays below single-intent R@1 for every method: a multi-intent request multiplies decomposition quality by per-sub-query retrieval accuracy, so rank errors compound across sub-queries. The gap between R@5 and R@1 persists in both halves — the correct resource is usually retrieved but not ranked first — which is why a per-sub-query re-ranking step is the most direct path to higher end-to-end accuracy. Figure 9 shows this single-to-multi drop per method.

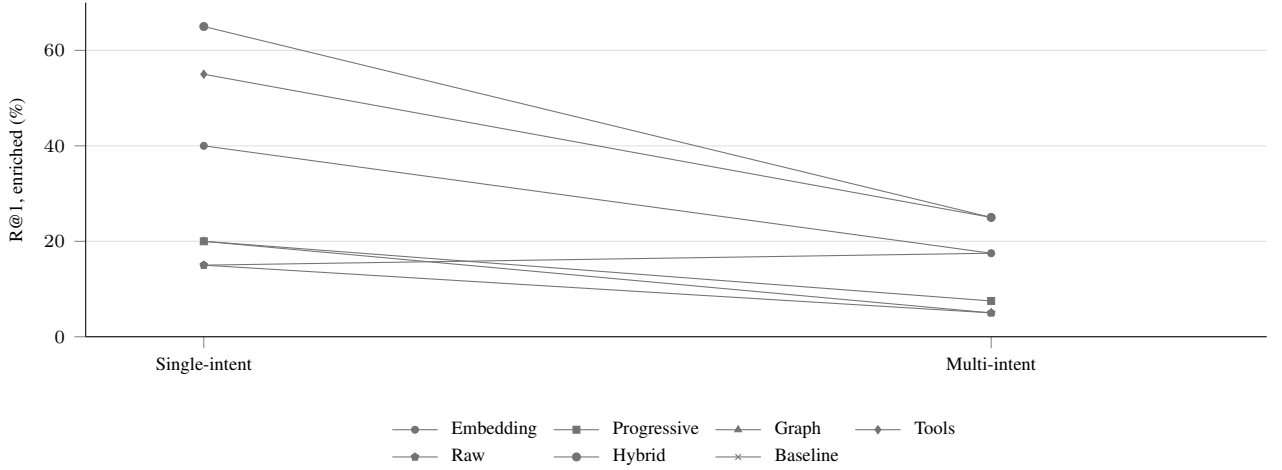


Figure 9: R@1 drop from single-intent to multi-intent Dynamic cases under enrichment. Every retrieval method degrades sharply (H: 65 → 25, T: 55 → 25), because decomposing a multi-intent request multiplies per-sub-query errors. Only the no-retrieval baseline B is flat, as it never decomposes.

Table 14: Dynamic results ($n=40$: 20 single-intent + 20 multi-intent). Suffix 0 = without enrichment, 1 = with enrichment. Δ = enriched – baseline. Tokens = avg. per case. R@1, R@5, Δ R@1 in % (pp for Δ). Bold = best per column. [†]amortised embedding cost; *agentic composite (H).

Meth.	Combined				Single-Intent		Multi-Intent	
	R@1	R@5	Δ R@1	Tok.	R@1	R@5	R@1	R@5
Baseline ₀	16.3	46.3	—	23,688	15.0	50.0	17.5	42.5
Baseline ₁	16.3	46.3	± 0.0	23,688	15.0	50.0	17.5	42.5
Embedding ₀ [†]	32.5	67.5	—	693	45.0	80.0	20.0	55.0
Embedding ₁ [†]	28.7	70.0	−3.8	999	40.0	85.0	17.5	55.0
Progressive ₀	13.8	28.7	—	5,773	15.0	30.0	12.5	27.5
Progressive ₁	13.8	27.5	± 0.0	6,292	20.0	30.0	7.5	25.0
Graph ₀	2.5	7.5	—	4,405	5.0	15.0	0.0	0.0
Graph ₁	12.5	17.5	+10.0	7,359	20.0	20.0	5.0	15.0
Tools ₀	25.0	31.2	—	39,760	35.0	45.0	15.0	17.5
Tools ₁	40.0	48.7	+15.0	47,859	55.0	70.0	25.0	27.5
Raw ₀	12.5	12.5	—	71,235	25.0	25.0	0.0	0.0
Raw ₁	10.0	10.0	−2.5	52,117	15.0	15.0	5.0	5.0
Hybrid ₀ [*]	42.5	51.2	—	17,583	50.0	50.0	35.0	52.5
Hybrid ₁ [*]	45.0	53.7	+2.5	22,969	65.0	65.0	25.0	42.5